Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

FCEFyN

Procesamiento Digital de Señales

Ingeniería en Computación

Laboratorio 4

Filtros adaptativos LMS en MCXN947

FRDM-MCXN947 · MCUXpresso IDE · CMSIS-DSP · MATLAB

Integrante	Matrícula	Correo
Costamagna, Matias Javier	44.512.437	matias.costamagna@mi.unc.edu.ar
Moreyra, Julián	45.214.537	julian.moreyra@mi.unc.edu.ar
Rivera, Mariano	45.432.921	luismarianorivera.25@mi.unc.edu.ar
Rojas, Facundo Nicolás	44.629.222	facundo.rojas.222@mi.unc.edu.ar

Docentes: Rossi, Roberto Raúl · Parlanti, Gustavo Fernando · Molina, Germán Alcides · Ferreyra, Pablo Alejandro

Fecha de entrega: 12 de noviembre de 2026

Índice

1. Introducción **2**

2. Marco teórico **2**

2.1. Combinador lineal adaptativo y filtro transversal 2

2.2. Máxima pendiente y LMS 2

2.3. LMS normalizado 3

2.4. Esquemas usados 3

3. Materiales y conexionado **3**

4. Diseño de la aplicación **4**

4.1. Organización del firmware 4

4.2. Parte 1: identificación de planta 5

4.3. Parte 2: cancelador de ruido 5

4.4. Interfaz 6

5. Resultados de la Parte 1 **6**

5.1. Influencia de μ con entrada aleatoria 7

5.2. Entrada constante 8

5.3. Influencia de la potencia de entrada 9

5.4. Ruido de medición y desajuste 10

5.5. Comparación de implementaciones 11

5.6. Resultados en la placa 12

6. Resultados de la Parte 2 **13**

6.1. Resultados en la placa 15

7. Conclusiones **16**

A. Código fuente **18**

1 Introducción

Laboratorio 4 — Filtros adaptativos

Objetivo. Verificar el funcionamiento de los sistemas adaptativos e implementarlos en los DSP de la familia Cortex M4.
Parte 1: análisis de un proceso adaptativo. Realizar un programa en C que muestre el proceso adaptativo de un filtro transversal de 30 coeficientes, utilizando el algoritmo LMS y estudiando el mismo con diferentes valores de la constante de velocidad de adaptación (μ), en el esquema de identificación de planta. Simular una planta de entrada única con una respuesta al impulso de 30 muestras con valores arbitrarios fijos; calcular el MSE promediándolo cada 100 muestras y mostrar en ese momento la diferencia entre los coeficientes de la planta y los del filtro adaptativo; usar como entrada una señal pseudorandom y repetir con una entrada constante. Analizar la influencia de μ y de la potencia de entrada.
Parte 2: análisis de la influencia del ruido. Utilizar el mismo FIR de 30 coeficientes para correlacionar la señal random (ruido) con la entrada de referencia del procesador adaptativo. Ingresar a la placa una senoidal y sumarle el ruido. Demostrar que el procesador adaptativo converge al filtro óptimo que cancela el ruido de la señal+ruido y mostrar la señal con un osciloscopio.

La cátedra propone la FRDM-K64F (Cortex-M4); el trabajo se hizo en la FRDM-MCXN947, cuyo núcleo Cortex-M33 tiene la misma FPU de simple precisión y las mismas instrucciones DSP, y corre la misma biblioteca CMSIS-DSP.

Todo el código está en la carpeta `TP4/` del repositorio

`https://github.com/Mati-Costamagna/procesamiento-digital-de-senales`

Ahí están el firmware en `TP4/source`, los scripts de MATLAB en `TP4/matlab` y una simulación en la PC que corre exactamente el mismo código C en `TP4/tools/host_sim`.

2 Marco teórico

2.1 Combinador lineal adaptativo y filtro transversal

Con una entrada única, el vector de entrada son las últimas L muestras, $\mathbf{X}_n = [x_n, \dots, x_{n-L+1}]^T$, y la salida es

$$y_n = \mathbf{W}^T \mathbf{X}_n = \sum_{k=0}^{L-1} w_k \, x_{n-k}, \qquad e_n = d_n - y_n. \tag{1}$$

Si las señales son estacionarias, el error cuadrático medio es una función cuadrática de los pesos,

$$\xi = E[e_n^2] = E[d_n^2] - 2\mathbf{P}^T\mathbf{W} + \mathbf{W}^T\mathbf{R}\mathbf{W}, \qquad \mathbf{R} = E[\mathbf{X}_n\mathbf{X}_n^T], \quad \mathbf{P} = E[d_n\mathbf{X}_n], \tag{2}$$

con un único mínimo en el vector de Wiener $\mathbf{W}^* = \mathbf{R}^{-1}\mathbf{P}$, donde $\xi_{min} = E[d_n^2] - \mathbf{P}^T\mathbf{W}^*$.

2.2 Máxima pendiente y LMS

El método de máxima pendiente itera en contra del gradiente, $\mathbf{W}_{k+1} = \mathbf{W}_k - \mu_t \nabla_k$. Trasladando y rotando los ejes a los vectores propios de $\mathbf{R}$, cada modo p evoluciona en forma independiente como $(1 - 2\mu_t\lambda_p)^k$, así que converge si $0 < \mu_t < 1/\lambda_{max}$. Como los autovalores no se conocen, se usa la cota más segura $0 < \mu_t < 1/\mathrm{tr}(\mathbf{R})$, con $\mathrm{tr}(\mathbf{R}) = L\,P_x$ para un filtro transversal ($P_x = E[x_n^2]$).

El LMS reemplaza el gradiente por su estimación instantánea $\hat{\nabla}_k = -2e_k\mathbf{X}_k$:

$$\mathbf{W}_{k+1} = \mathbf{W}_k + 2\mu_t\, e_k\mathbf{X}_k. \tag{3}$$

No necesita calcular $\mathbf{R}$ ni sus autovalores, ni esperar a juntar muestras: adapta en cada muestra. En valor medio sigue la misma trayectoria que máxima pendiente, y una vez convergido queda "ruido de gradiente" alrededor de $\mathbf{W}^*$.

Convención usada en el código. CMSIS-DSP (y la rutina `LMSfir` de la clase) escriben la actualización como $w_k \leftarrow w_k + \mu\, e_n\, x_{n-k}$, es decir $\mu = 2\mu_t$. En todo el informe se usa el μ de CMSIS, con lo que las fórmulas quedan:

convergencia: $$0 < \mu < \frac{2}{\lambda_{max}} \;\Rightarrow\; 0 < \mu < \frac{2}{L\,P_x} \quad \text{(cota segura)} \tag{4}$$

constante de tiempo del MSE: $$\tau_{mse,p} \approx \frac{1}{2\mu\lambda_p} \text{ muestras} \tag{5}$$

desajuste: $$M = \frac{\xi_{exceso}}{\xi_{min}} \approx \frac{\mu\,\mathrm{tr}(\mathbf{R})/2}{1-\mu\,\mathrm{tr}(\mathbf{R})/2} \approx \frac{\mu}{2}LP_x \tag{6}$$

2.3 LMS normalizado

En el NLMS (`arm_lms_norm_f32`) el paso se divide por la energía de la entrada en la ventana del filtro, $E_n = \sum_{k=0}^{L-1} x_{n-k}^2 \approx LP_x$: $w_k \leftarrow w_k + \mu\, e_n x_{n-k}/E_n$. El paso efectivo es $\mu/(LP_x)$, así que la condición $0 < \mu < 2$ y la velocidad de convergencia ya no dependen de la potencia de la entrada.

2.4 Esquemas usados

Identificación de planta: la entrada x excita a la planta $P(z)$ y al filtro adaptativo; la salida de la planta es la señal deseada d. Si la entrada es suficientemente rica, $\mathbf{W}^*$ es la respuesta al impulso de la planta. **Cancelador de interferencia:** $d = s + n'$, donde la señal útil s está contaminada con un ruido n', y el filtro recibe una referencia x distinta pero correlacionada con n' y no con s. El filtro estima n' a partir de x y el error $e = s + n' - y$ tiende a s.

3 Materiales y conexionado

Tabla 1: Equipamiento.

Placa	NXP FRDM-MCXN947 (MCXN947, Cortex-M33 a 150 MHz, FPU y DSP)
Entorno	MCUXpresso IDE v11.10.0, SDK FRDM-MCXN947, CMSIS-DSP
PC	MATLAB (R2019b o posterior) y terminal serie a 115200 baud
Generador de funciones	[modelo]
Osciloscopio	[modelo]

Tabla 2: Conexiones de la Parte 2 (la Parte 1 sólo usa el USB).

Señal	Pin MCU	Conector	Uso
Entrada analógica (ADC0, canal A2)	P4_23	J8 pin 28	senoidal del generador
Salida DAC0	P4_2	J1 pin 4	canal 2 del osciloscopio
TRIG (pulso al reiniciar **W**)	P1_23	J1 pin 8	disparo externo
PROC (en alto mientras procesa)	P4_0	J2 pin 18	tiempo de cómputo
GND	–	cualquier GND	masa común
Puerto serie (LPUART4)	P1_8 / P1_9	USB J17 (MCU-Link)	VCOM

El ADC mide de 0 a 3,3 V, así que la senoidal se monta sobre 1,65 V de continua (1 Vpp, salida del generador en High-Z). El firmware estima y resta esa continua con un filtro de seguimiento lento ($\alpha = 2^{-12}$), y el DAC saca la señal elegida centrada en 1,65 V.

4 Diseño de la aplicación

4.1 Organización del firmware

Las dos partes van en un único programa; se pasa de una a otra con una pulsación larga de SW3 o con los comandos `1` y `2` por el puerto serie. El procesamiento está separado del hardware: `parte1.c` y `parte2.c` sólo reciben y devuelven números, y `main.c` hace de "placa" (teclas, UART, ADC, DAC, timer). Así el mismo código se compila también en la PC (`TP4/tools/host_sim`), lo que permitió verificar los algoritmos y generar las figuras simuladas de este informe antes de ir al laboratorio.

Tabla 3: Módulos del firmware.

`lms.c/.h`	Filtro adaptativo con tres implementaciones intercambiables: la rutina `LMSfir` de la clase, `arm_lms_f32` y `arm_lms_norm_f32`.
`parte1.c/.h`	Identificación de planta: entrada, planta, LMS, MSE cada 100 muestras.
`parte2.c/.h`	Cancelador: ruido, correlador, LMS, estadísticas por bloque.
`coefs.c/.h`	Coeficientes fijos de la planta y del correlador.
`fir_fijo.h`	FIR de coeficientes fijos con buffer circular.
`prng.h`	Generador pseudoaleatorio xorshift32 (uniforme en $[-1, 1)$).
`txt_out.c/.h`	Arma las líneas CSV con floats sin depender de `printf`.
`main.c`	Periféricos, teclas, comandos, ISR de muestreo.
`app_config.h`	Todos los parámetros ajustables.

Generador pseudoaleatorio. Se usa un xorshift32 propio en lugar de `rand()` para que la secuencia sea la misma en la placa y en la PC, y para repetir un experimento con la misma entrada cambiando sólo μ (todas las corridas de un barrido usan la semilla 1). Con 24 bits de mantisa se obtiene una uniforme en $[-A, A)$, de media nula y $P_x = A^2/3$.

Implementaciones del LMS. La rutina de la diapositiva "Implementación en DSP" se pasó a `lms_clase_step()`: desplaza la línea de retardo, calcula y y adapta. La única diferencia es que el error $d - y$ se calcula una vez, con la salida completa de la muestra actual, antes de adaptar; en la versión de la diapositiva que funde los tres lazos, el $(d - y)$ usa la y parcial que se va acumulando. Las funciones de CMSIS se llaman con `blockSize = 1` (muestra a muestra) y guardan los coeficientes en orden invertido en el tiempo, por lo que `lms_get_w()` los da vuelta para compararlos con la planta.

4.2 Parte 1: identificación de planta

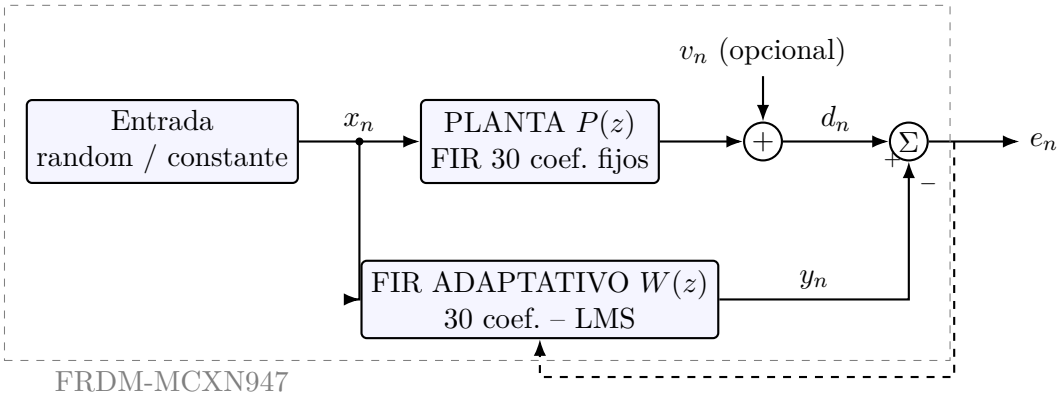


Figura 1: Esquema de identificación de planta implementado (figura 1 de la consigna). Todo corre dentro del microcontrolador.

La planta tiene una respuesta al impulso de 30 muestras con valores arbitrarios fijos, $p_k = 0{,}8\,e^{-k/8}\sin(0{,}3\pi(k+1)) + 0{,}05\cos(0{,}9k)$ redondeados a 4 decimales ($\sum p_k^2 = 1{,}979$). Cada experimento procesa $N = 20000$ muestras en bloques de 100. Al final de cada bloque se envía por el puerto serie:

- `P1M,n,mse,werr2,mse_dB`: el MSE promedio de las 100 muestras, $\frac{1}{100}\sum e_n^2$, y el error de los coeficientes $\|\mathbf{p} - \mathbf{w}\|^2$;
- `P1D,n,d0,...,d29`: la diferencia $p_k - w_k$ de cada uno de los 30 coeficientes.

Al principio se manda la configuración (`P1I`, que incluye μ_{max} y τ teóricos) y la planta (`P1P`); al final los coeficientes aprendidos (`P1W`) y los ciclos de CPU promedio por muestra del LMS medidos con el contador `DWT->CYCCNT` (`P1F`). Si el MSE supera 10^{12} la corrida se corta y se informa `DIVERGE`.

4.3 Parte 2: cancelador de ruido

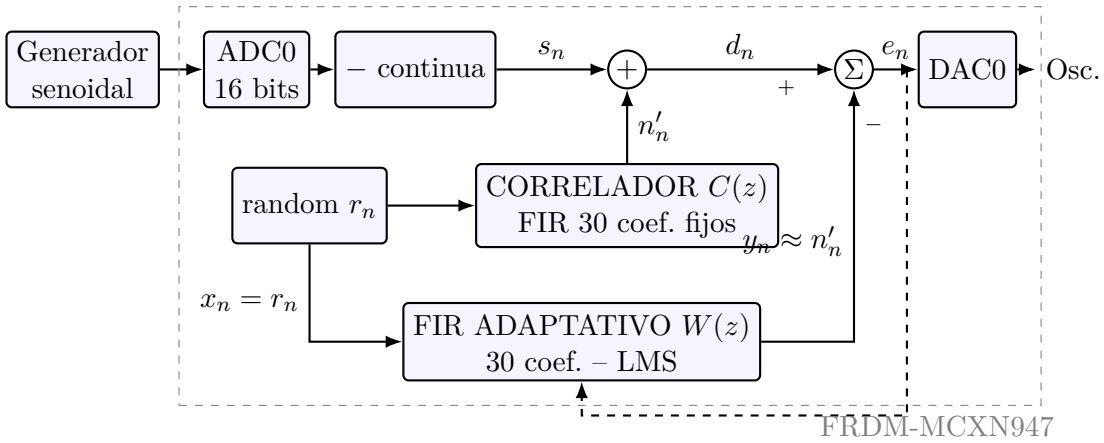


Figura 2: Cancelador implementado. El ruido r_n se genera adentro de la placa, pasa por el correlador y se suma a la senoidal; el filtro adaptativo recibe r_n como referencia. Por el DAC se puede sacar d, e, y, s o el residuo $e - s$.

El CTIMER0 marca $f_s = 8$ kHz y en cada período dispara una conversión del ADC. Toda la cadena de la figura 2 corre en la interrupción del ADC, muestra a muestra:

1. se convierte la lectura a volts y se resta la continua: s_n;
2. se genera r_n uniforme en $[-0{,}4, 0{,}4]$ V y se filtra con el correlador: $n'_n = C\{r\}_n$;
3. $d_n = s_n + n'_n$, y el LMS con referencia $x_n = r_n$ da y_n y $e_n = d_n - y_n$;
4. la señal elegida se escribe en el DAC.

El correlador es $c_k = 0{,}6 \cdot 0{,}8^k$, $k = 0 \dots 29$ (energía $\sum c_k^2 = 1$, así que el ruido mantiene su potencia: $P_{n'} = P_r = 0{,}4^2/3 = 0{,}0533$ V^2). Como r_n es blanco, $\mathbf{R} = P_r\mathbf{I}$ y $\mathbf{P} = E[d_n\mathbf{X}_n] = P_r\,\mathbf{c}$, de modo que el filtro óptimo es exactamente $\mathbf{W}^* = \mathbf{c}$: el procesador tiene que *copiar* al correlador. Con eso $y = n'$ y $e = s$.

Como el ruido se genera en la placa, también se conoce s_n limpia, y se puede medir cuánto ruido queda a la salida: el residuo $e_n - s_n$. Cada 100 muestras se envía una línea `P2M` con la potencia del ruido, la del residuo, la reducción en dB, $\|\mathbf{W}^* - \mathbf{w}\|^2$ y las SNR de entrada y salida. Se usa $\|\mathbf{W}^* - \mathbf{w}\|^2$ como medida de convergencia al óptimo. SW2 pone los coeficientes en cero y genera un pulso en TRIG, para disparar el osciloscopio justo cuando el filtro empieza a aprender.

Ubicación del correlador. La figura 2 de la consigna dibuja el correlador en la rama de la referencia: $d = s + r$ y $x' = C\{r\}$. En ese caso el óptimo es $\mathbf{W}^* = C^{-1}$, que existe y es estable sólo si $C(z)$ es de fase mínima. Se eligió un correlador de fase mínima (sus ceros están en $|z| = 0{,}8$) y casi igual a $0{,}6/(1 - 0{,}8z^{-1})$, de modo que $C^{-1}(z) \approx (1 - 0{,}8z^{-1})/0{,}6$: un FIR de dos coeficientes, $[1{,}667;\ -1{,}333]$. Las dos variantes están implementadas (comando `Q0` / `Q1`); por defecto se usa la del correlador en el camino del ruido, que es la del cancelador de interferencia de la teoría.

4.4 Interfaz

Tabla 4: Teclas y comandos (letra + valor + Enter, desde cualquier terminal o MATLAB).

SW3 largo / `1`, `2`	Cambia entre Parte 1 (LED verde) y Parte 2 (LED rojo).
SW2	P1: arranca (o corta) un experimento. P2: reinicia los coeficientes + pulso en TRIG.
SW3 corto	P1: siguiente μ de la lista. P2: cambia la señal del DAC (d, e, y, s, residuo).
`U<mu>`, `G<0|1|2>`	Paso de adaptación; algoritmo (clase, `arm_lms_f32`, NLMS).
`A<amp>`	P1: amplitud de la entrada. P2: amplitud del ruido.
`E<0|1>`, `V<sv>`, `S`, `B`	P1: entrada aleatoria/constante, ruido de medición, arrancar, barrido de μ.
`Q`, `O`, `Z`, `C`, `T`, `F`	P2: topología, salida del DAC, reinicio, coeficientes, tiempos, f_s.

5 Resultados de la Parte 1

Las figuras de esta sección se generaron corriendo en la PC el mismo código del firmware (`p1_sim`), con la misma CMSIS-DSP. En la placa se repiten con `tp4_experimentos.m`, que manda los mismos comandos y produce las mismas figuras (sección 5.6).

5.1 Influencia de μ con entrada aleatoria

Con $A = 1$, $P_x = 1/3$ y la cota (4) da $\mu_{max} = 2/(30 \cdot 1/3) = 0{,}2$. Como la entrada es blanca, $\mathbf{R} = P_x\mathbf{I}$: los 30 autovalores son iguales y (5) da $\tau_{mse} = 1/(2\mu P_x) = 1{,}5/\mu$ muestras.

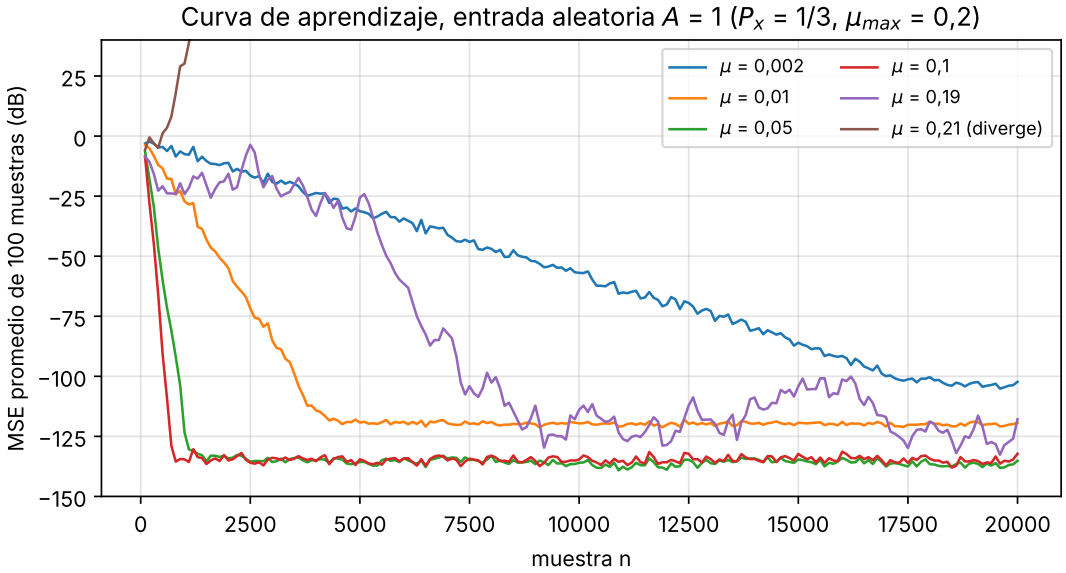


Figura 3: Curva de aprendizaje (MSE promediado cada 100 muestras) para distintos μ.

- La pendiente de la curva en dB es $-10\log_{10}(e)/\tau \approx -4{,}34/\tau$ dB por muestra. Para $\mu = 0{,}01$ ($\tau = 150$) se esperan 29 dB cada 1000 muestras, y la curva baja de -3 a -55 dB en las primeras 2000: coincide.
- Sin ruido de medición $\xi_{min} = 0$, así que el MSE baja hasta el piso de la aritmética de punto flotante de simple precisión (-120 a -135 dB) y los coeficientes llegan exactamente a los de la planta (figura 5b).
- Hasta $\mu \approx 0{,}05$–$0{,}1$, aumentar μ acelera el aprendizaje. Más cerca del límite ($\mu = 0{,}19$) el proceso se vuelve errático y más *lento*, y con $\mu = 0{,}21 > \mu_{max}$ diverge. La cota $2/\mathrm{tr}(\mathbf{R})$ resultó muy ajustada: para el LMS (no para máxima pendiente, que con autovalores iguales admitiría hasta $2/P_x = 6$) el límite real de estabilidad en media cuadrática está cerca de $2/\mathrm{tr}(\mathbf{R})$, porque el ruido del gradiente crece con $\mu\,\mathrm{tr}(\mathbf{R})$.

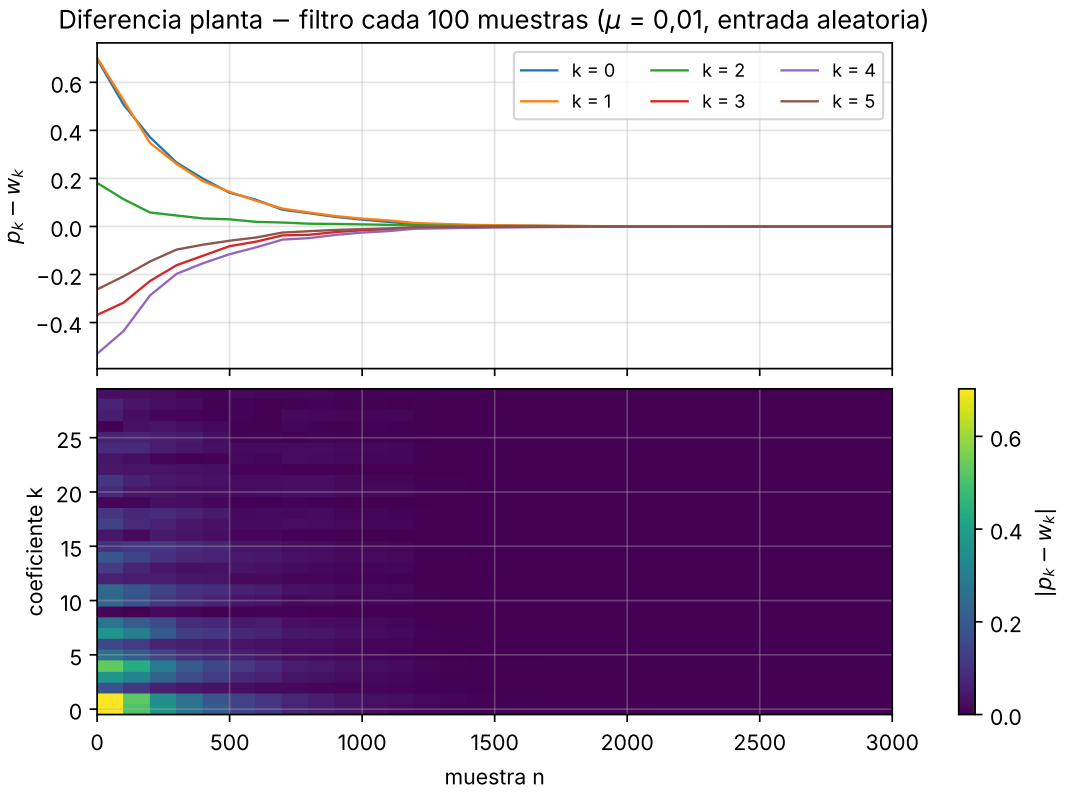
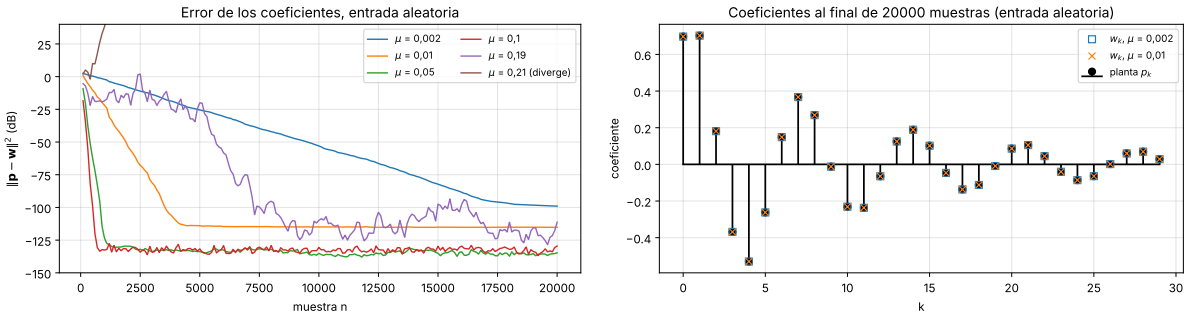


Figura 4: Diferencia entre los coeficientes de la planta y los del filtro, mostrada cada 100 muestras ($\mu = 0{,}01$). Arriba, los 6 primeros coeficientes; abajo, $|p_k - w_k|$ de los 30.



(a) $\|\mathbf{p} - \mathbf{w}\|^2$.

(b) Coeficientes finales.

Figura 5: Error de los coeficientes y coeficientes aprendidos con entrada aleatoria.

5.2 Entrada constante

Con $x_n = A$, $\mathbf{R} = A^2\mathbf{1}\mathbf{1}^T$ tiene un solo autovalor no nulo, $\lambda = LA^2 = \mathrm{tr}(\mathbf{R})$, con vector propio $\mathbf{1}$, y 29 autovalores nulos. El MSE sólo "ve" la componente de $\mathbf{w}$ en la dirección de $\mathbf{1}$, es decir, la suma de los coeficientes (la ganancia en continua). En consecuencia:

- El MSE converge a cero muy rápido ($\tau = 1/(2\mu LA^2)$, menos de 10 muestras), pero sólo porque $\sum w_k = \sum p_k$ (la ganancia de continua): figura 7.
- Los coeficientes **no** se identifican: $\|\mathbf{p} - \mathbf{w}\|^2$ queda cerca de $\|\mathbf{p}\|^2$. Los modos con $\lambda = 0$ no se adaptan nunca. Es la "solución parcial por falta de excitación apropiada" de la teoría.

(Los primeros coeficientes se mueven un poco porque al arrancar la línea de retardo se llena desde cero, lo que equivale a un escalón con algo de excitación.)

- La cota de estabilidad es la misma fórmula, $\mu < 2/(LA^2) = 0{,}0667$: con $\mu = 0{,}07$ diverge. La entrada constante tiene el triple de potencia que la aleatoria de la misma amplitud, y por eso diverge con un μ tres veces menor.

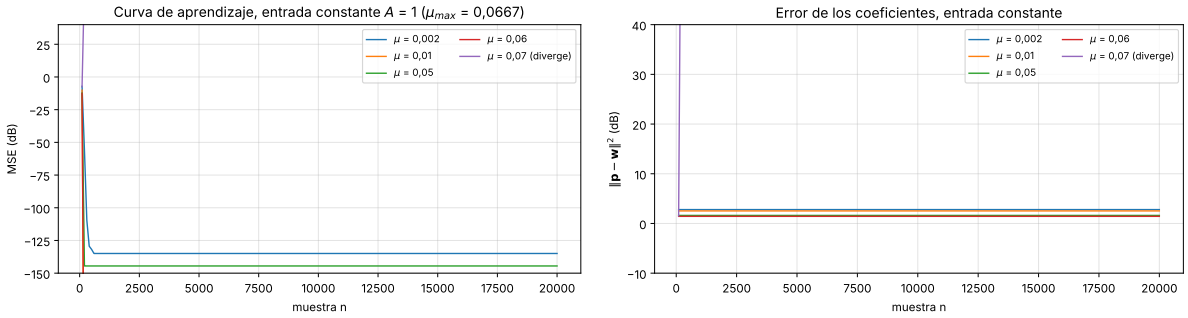


Figura 6: Entrada constante: el MSE llega al piso numérico pero el error de los coeficientes no baja.

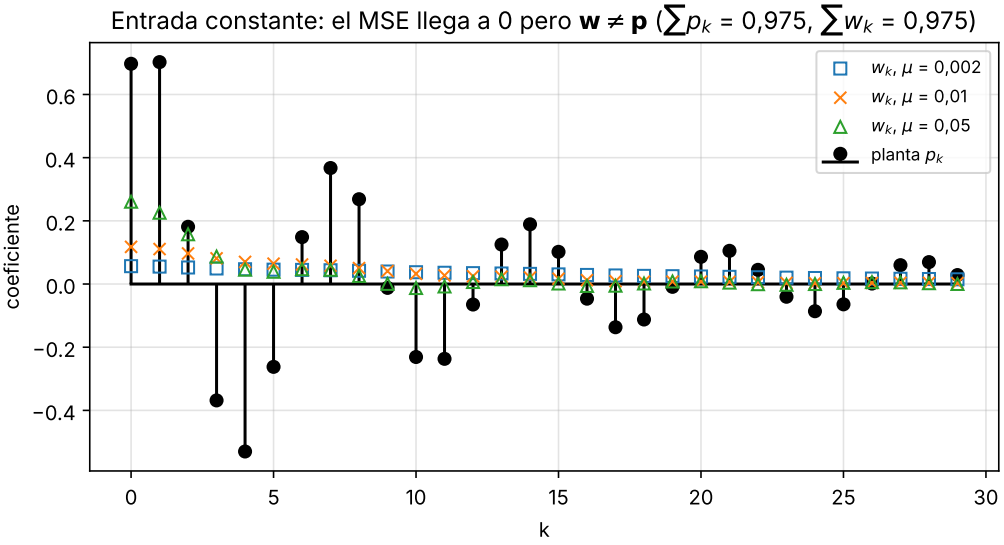


Figura 7: Con entrada constante el filtro aprende una respuesta distinta de la planta pero con la misma ganancia de continua.

5.3 Influencia de la potencia de entrada

Con el LMS, μ_{max} y τ dependen de P_x: con $\mu = 0{,}01$ fijo, duplicar la amplitud multiplica P_x por 4, divide τ por 4 (más rápido) y divide μ_{max} por 4. Con $A = 5$, $\mu_{max} = 0{,}008 < 0{,}01$ y el filtro diverge. El NLMS normaliza el paso con la energía de la entrada: las cuatro curvas son prácticamente iguales (figura 8). El piso numérico sube con A porque crece la escala de las señales.

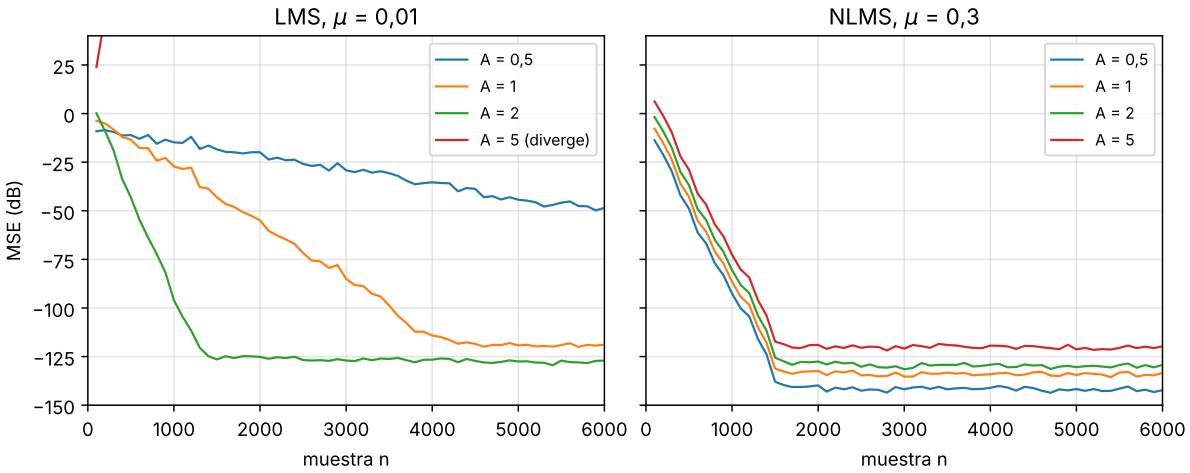


Figura 8: Misma planta y mismo μ con distintas amplitudes de entrada: LMS (izquierda) y NLMS (derecha).

5.4 Ruido de medición y desajuste

Si a la salida de la planta se suma un ruido v_n de varianza σ_v^2 (comando `V`), $\xi_{min} = \sigma_v^2$ y el MSE en régimen es $\sigma_v^2(1 + M)$. Con $\sigma_v = 0{,}01$ (-40 dB) aparece el compromiso típico: un μ más grande converge más rápido pero deja más ruido de gradiente. Los valores medidos en las últimas 5000 muestras siguen a (6) (tabla 5); el pequeño exceso es la variabilidad de estimar una potencia con bloques de 100 muestras.

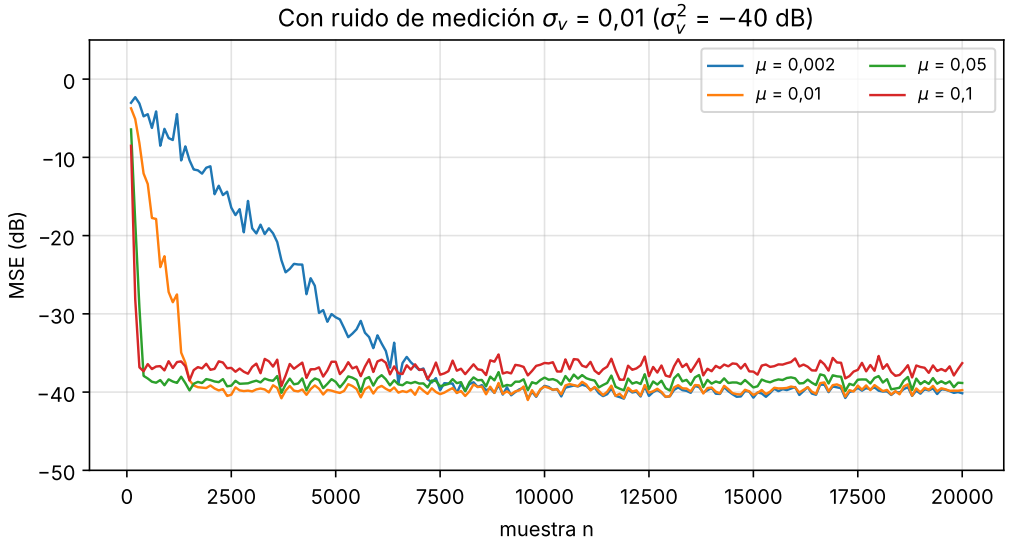


Figura 9: Curvas de aprendizaje con ruido de medición $\sigma_v = 0{,}01$.

Tabla 5: Desajuste con $\sigma_v = 0{,}01$, entrada aleatoria $A = 1$ (simulación).

μ	M teórico	MSE teórico	MSE medido	M medido
0,002	0,010	1,010e-04	1,043e-04	0,043
0,01	0,053	1,053e-04	1,088e-04	0,088
0,05	0,333	1,333e-04	1,372e-04	0,372
0,1	1,000	2,000e-04	2,076e-04	1,076

5.5 Comparación de implementaciones

La rutina de la clase y `arm_lms_f32` dan curvas idénticas bit a bit en la simulación (hacen las mismas operaciones en el mismo orden). El NLMS con $\mu = 0{,}3$ equivale a un LMS con $\mu = 0{,}3/(LP_x) = 0{,}03$. En la placa se mide el costo de cada una (tabla 7).

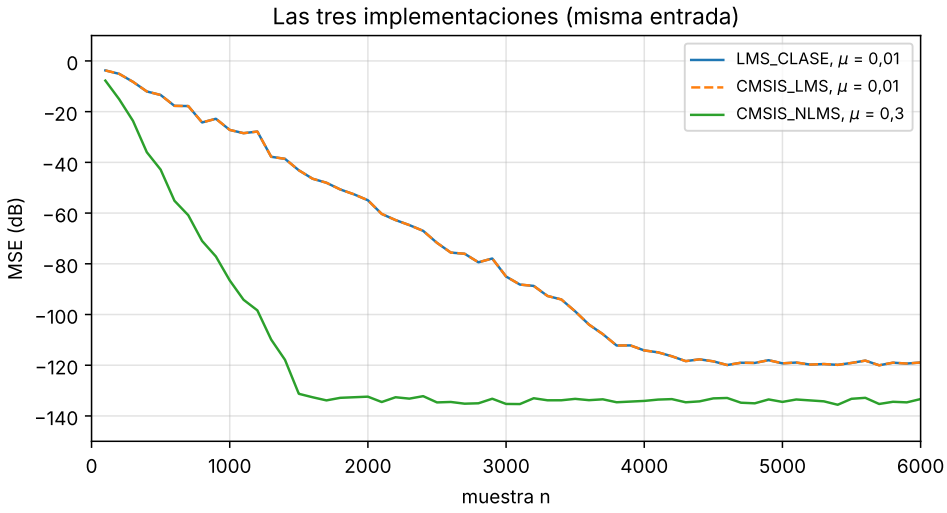


Figura 10: Las tres implementaciones con la misma entrada.

Tabla 6: Resumen de las corridas de la Parte 1 (simulación). τ en muestras.

Entrada	Alg.	A	μ	μ_{max}	τ teó.	MSE final	Estado
random	CMSIS_LMS	1	0,002	0,2	750	5,89e-11	ok
random	CMSIS_LMS	1	0,01	0,2	150	1,15e-12	ok
random	CMSIS_LMS	1	0,05	0,2	30	3,04e-14	ok
random	CMSIS_LMS	1	0,1	0,2	15	6,01e-14	ok
random	CMSIS_LMS	1	0,19	0,2	7,89	1,64e-12	ok
random	CMSIS_LMS	1	0,21	0,2	7,14	1,24e+12	diverge
const	CMSIS_LMS	1	0,002	0,0667	8,33	3,20e-14	ok
const	CMSIS_LMS	1	0,01	0,0667	1,67	0,00e+00	ok
const	CMSIS_LMS	1	0,05	0,0667	0,333	3,55e-15	ok
const	CMSIS_LMS	1	0,06	0,0667	0,278	0,00e+00	ok
const	CMSIS_LMS	1	0,07	0,0667	0,238	4,45e+15	diverge
pot	CMSIS_LMS	0,5	0,01	0,8	600	6,03e-12	ok
pot	CMSIS_LMS	1	0,01	0,2	150	1,15e-12	ok
pot	CMSIS_LMS	2	0,01	0,05	37,5	1,66e-13	ok
pot	CMSIS_LMS	5	0,01	0,008	6	6,66e+12	diverge
pot	CMSIS_NLMS	0,5	0,3	2	50	8,58e-15	ok
pot	CMSIS_NLMS	1	0,3	2	50	4,07e-14	ok
pot	CMSIS_NLMS	2	0,3	2	50	1,36e-13	ok
pot	CMSIS_NLMS	5	0,3	2	50	1,22e-12	ok

5.6 Resultados en la placa

Captura pendiente: **Curva de aprendizaje medida (tp4_experimentos.m, sección E1)**
guardar como `img/placa/p1_e1_random_mse.png`

Figura 11: Curva de aprendizaje medida en la FRDM-MCXN947, entrada aleatoria.

Captura pendiente: **Coeficientes con entrada constante (sección E2)**
guardar como `img/placa/p1_e2_const_coef.png`

Figura 12: Coeficientes finales medidos con entrada constante.

Captura pendiente: **Captura de la terminal con las líneas P1M / P1D**
guardar como `img/placa/terminal_p1.png`

Figura 13: Programa corriendo: salida por el puerto serie durante un experimento.

Tabla 7: Ciclos de CPU por muestra del filtro adaptativo ($L = 30$, 150 MHz). Completar con la línea `P1F` de cada corrida de la sección E4.

Implementación	Ciclos/muestra	Tiempo (μs)
LMS de clase (`lms_clase_step`)	[]	[]
`arm_lms_f32`	[]	[]
`arm_lms_norm_f32`	[]	[]

6 Resultados de la Parte 2

Simulación con una senoidal de 200 Hz y 1 Vpp, cuantizada a 16 bits, $f_s = 8$ kHz, ruido de amplitud 0,4 V ($P_r = 0{,}0533$ V^2, SNR de entrada $\approx 3{,}7$ dB). Para la referencia, $\mu_{max} = 2/(30 \cdot 0{,}0533) = 1{,}25$ y $\tau = 1/(2\mu P_r)$.

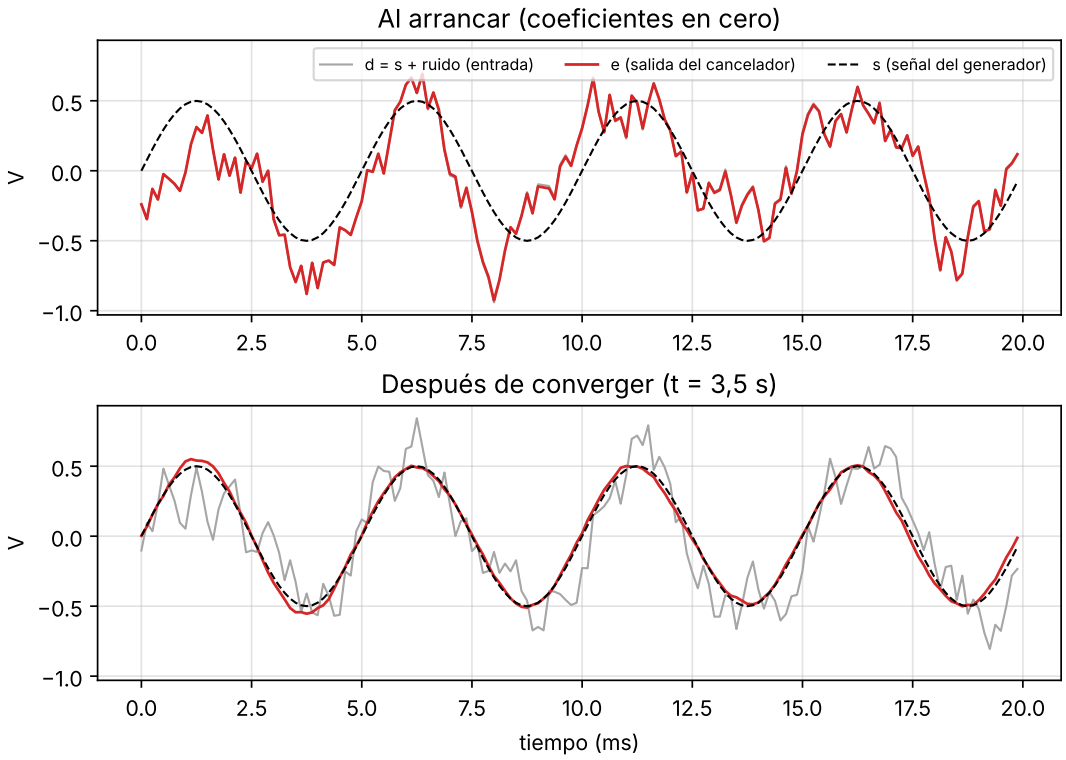


Figura 14: Formas de onda (osciloscopio simulado) con $\mu = 0{,}005$: al arrancar la salida es igual a la entrada ruidosa; a los 3,5 s el cancelador recupera la senoidal.

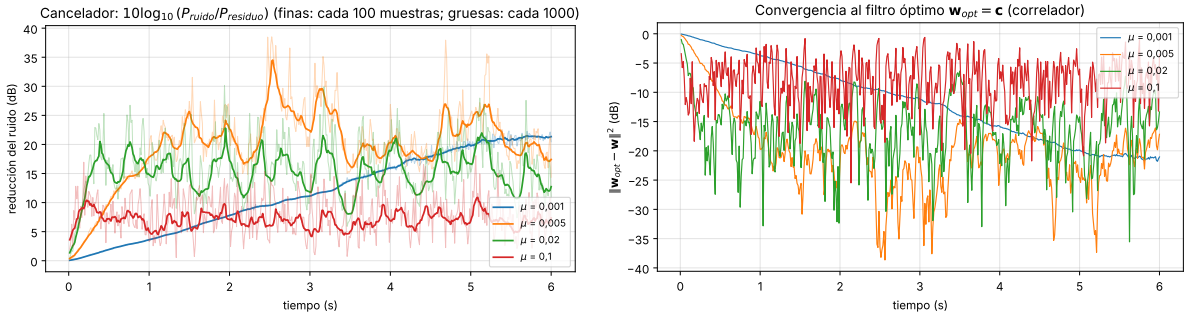


Figura 15: Reducción de ruido y distancia al filtro óptimo $\mathbf{W}^* = \mathbf{c}$ para distintos μ.

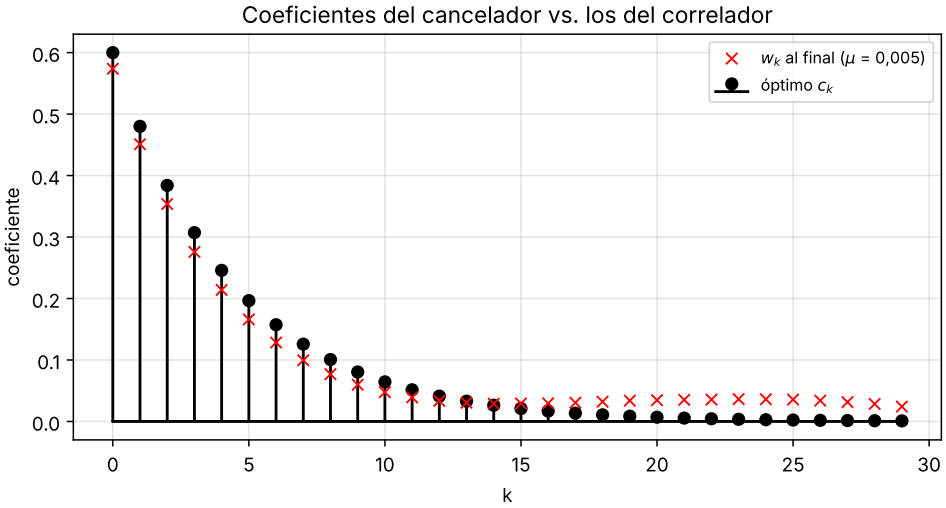


Figura 16: Coeficientes del cancelador después de 6 s ($\mu = 0{,}005$): convergen al correlador.

- El procesador converge al filtro óptimo de Wiener: los coeficientes copian al correlador (figura 16) y $\|\mathbf{W}^* - \mathbf{w}\|^2$ cae 20 dB o más (figura 15).
- Acá $\xi_{min} = P_s = 0{,}125$ V^2 no es cero: **la propia señal útil hace de ruido para el algoritmo**, porque $e = s + (n' - y)$ y el LMS adapta con todo e. Por eso el desajuste deja un ruido residual $\approx M\,P_s$ y aparece el compromiso velocidad–calidad: con $\mu = 0{,}1$ converge en decenas de milisegundos pero la reducción no pasa de 7–8 dB; con $\mu = 0{,}001$ tarda varios segundos pero supera los 20 dB (tabla 8).
- Con el correlador en la rama de la referencia (figura 17) el filtro también converge, a C^{-1}, pero mucho más lento. La referencia ahora es ruido coloreado, con densidad espectral $P_r|C(e^{j\omega})|^2$ que va de $9P_r$ (en $\omega = 0$) a $0{,}11P_r$ (en $\omega = \pi$): la dispersión de autovalores $\lambda_{max}/\lambda_{min} \approx 81$ hace que los modos lentos tarden 81 veces más que los rápidos, que fijan el μ máximo.

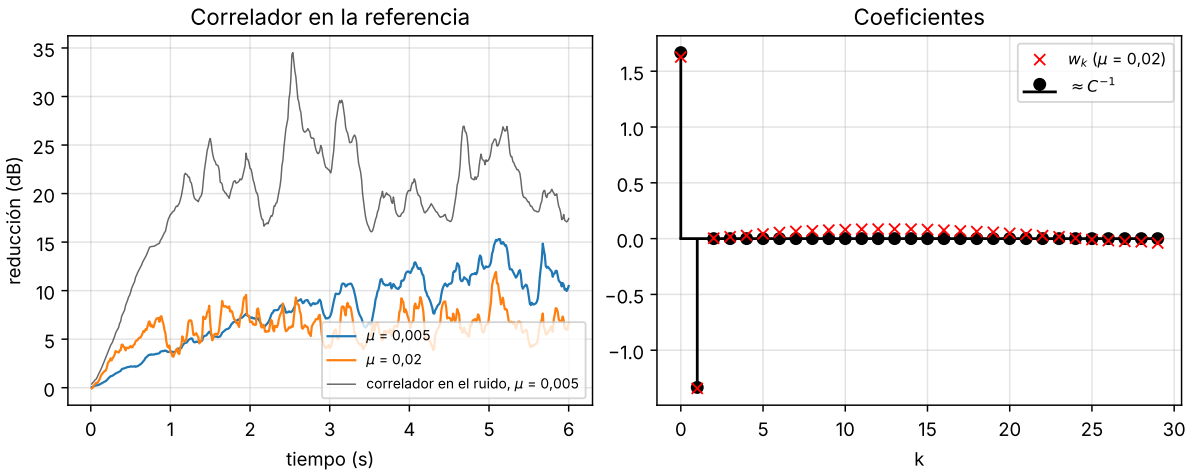


Figura 17: Correlador en la rama de la referencia (como en la figura 2 de la consigna).

Tabla 8: Cancelador en régimen (simulación, promedio de los últimos 0,25 s de 6 s).

Topología	μ	τ teó. (s)	Reducción (dB)	SNR salida (dB)
ruido	0,001	1,172	21,2	25,0
ruido	0,005	0,234	18,9	22,7
ruido	0,02	0,059	13,7	17,5
ruido	0,1	0,012	7,5	11,3
referencia	0,005	0,234	11,4	15,1
referencia	0,02	0,059	7,6	11,3

6.1 Resultados en la placa

Para ver la convergencia en el osciloscopio: CH1 en la salida del DAC (J1-4) con la señal e (comando `01`), disparo externo en TRIG (J1-8), modo *single*, y apretar SW2. Con $\mu = 0{,}005$ ($\tau \approx 0{,}23$ s) se ve cómo la senoidal se va limpiando en el primer segundo. Con `00` se ve la señal con ruido y con `04` el residuo.

Captura pendiente: **DAC = d (señal + ruido)**
guardar como `img/placa/osc_d.png`

(a) Entrada del cancelador, d.

Captura pendiente: **DAC = e, ya convergido**
guardar como `img/placa/osc_e.png`

(b) Salida del cancelador, e.

Captura pendiente: **DAC = e, disparo en TRIG al apretar SW2, base de tiempo 100–200 ms/div**
guardar como `img/placa/osc_convergencia.png`

(c) Aprendizaje después de reiniciar los coeficientes.

Figura 18: Capturas del osciloscopio.

Captura pendiente: **Reducción de ruido medida (tp4_experimentos.m, sección E6)**
guardar como `img/placa/p2_e6_mu_red.png`

Figura 19: Reducción de ruido medida en la placa.

Tiempo de cómputo. El comando `T` devuelve los ciclos promedio y máximo de la interrupción completa (ruido, correlador, LMS y DAC). El pin PROC (J2-18) está en alto durante ese tiempo, y en el osciloscopio se compara su ancho con el período de muestreo de 125 μs. [ciclos promedio y máximo de la línea P2T y ancho de PROC medido]

7 Conclusiones

- El LMS identifica una planta FIR de 30 coeficientes sin conocer nada de ella, sólo con su entrada y su salida. Con entrada aleatoria los coeficientes convergen exactamente a los de la planta, con la constante de tiempo que predice la teoría, $\tau = 1/(2\mu P_x)$.
- μ fija el compromiso central: más grande, más rápido, pero más ruido de gradiente (desajuste $M \approx \mu L P_x/2$) y, por encima de $2/(LP_x)$, divergencia. En la práctica el mejor μ está lejos del límite: cerca de él el aprendizaje es errático y más lento.
- La potencia de entrada cambia la escala de todo: con el LMS el mismo μ puede converger o diverger según la amplitud de la señal. El NLMS lo resuelve normalizando el paso, a costa de una división y de acumular la energía por muestra.
- La convergencia de los coeficientes requiere excitación: con entrada constante el MSE se anula pero sólo se aprende la ganancia de continua. Que el error sea chico no implica que el sistema esté identificado.

- En el cancelador el filtro converge al óptimo de Wiener, que es una copia del correlador, y recupera la senoidal. La señal útil actúa como ruido para el algoritmo, lo que obliga a usar μ chicos para lograr buena reducción. Cuando la referencia es coloreada, la dispersión de autovalores hace mucho más lenta la convergencia.
- Las tres implementaciones dan el mismo resultado; CMSIS-DSP es la recomendada en el Cortex-M por su costo en ciclos, y con 30 coeficientes el cancelador completo usa una fracción chica del período de muestreo.

A Código fuente

El proyecto completo está en la carpeta `TP4/` del repositorio

`https://github.com/Mati-Costamagna/procesamiento-digital-de-senales`

El firmware en `TP4/source/`, los scripts de MATLAB en `TP4/matlab/` y la simulación en la PC en `TP4/tools/host_sim/`. Se listan los archivos principales.

Código 1: `lms.h`

```c
/*
 * TP4 - Procesamiento Digital de Señales
 * lms.h: filtro transversal adaptativo (combinador lineal de entrada única).
 *
 * Tres implementaciones con la misma interfaz, para poder compararlas:
 *
 *   LMS_ALG_CLASE : la rutina LMSfir() de las diapositivas de clase,
 *                   escrita en C "a mano".
 *   LMS_ALG_CMSIS : arm_lms_f32() de CMSIS-DSP.
 *   LMS_ALG_NLMS  : arm_lms_norm_f32() de CMSIS-DSP (LMS normalizado).
 *
 * Las tres usan la misma convención que CMSIS-DSP y que la diapositiva:
 *
 *     y(n)    = sum_k w_k(n) x(n-k)
 *     e(n)    = d(n) - y(n)
 *     w_k(n+1) = w_k(n) + mu * e(n) * x(n-k)                 (LMS)
 *     w_k(n+1) = w_k(n) + mu * e(n) * x(n-k) / E(n)          (NLMS)
 *     E(n)    = x(n)^2 + x(n-1)^2 + ... + x(n-L+1)^2
 *
 * OJO: en la teoría (Widrow) el paso se escribe W(k+1) = W(k) + 2*mu*e*X,
 * así que el mu de este código equivale a 2*mu de la teoría. Con esta
 * convención la condición de convergencia queda 0 < mu < 2 / (L * Px).
 */
#ifndef LMS_H_
#define LMS_H_

#include <stdint.h>

#include "arm_math.h"

#define LMS_MAX_TAPS 32U

typedef enum
{
    LMS_ALG_CLASE = 0,
    LMS_ALG_CMSIS = 1,
    LMS_ALG_NLMS  = 2,
} lms_alg_t;

typedef struct
{
    lms_alg_t alg;
    uint16_t L;
    float32_t mu;

    /* LMS_ALG_CLASE: x_hist[0] = x(n), x_hist[1] = x(n-1), ...; w en orden natural */
    float32_t x_hist[LMS_MAX_TAPS];
    float32_t w[LMS_MAX_TAPS];

    /* CMSIS-DSP: coeficientes en orden invertido en el tiempo y estado de
     * L + blockSize - 1 muestras (acá blockSize = 1: muestra a muestra). */
    float32_t cm_coef[LMS_MAX_TAPS];
```

```
    float32_t cm_state[LMS_MAX_TAPS];
    arm_lms_instance_f32 cm;
    arm_lms_norm_instance_f32 nm;
} lms_t;

/* Inicializa con los coeficientes en cero */
void lms_init(lms_t *f, lms_alg_t alg, uint16_t L, float32_t mu);

/* Vuelve los coeficientes y la historia de la entrada a cero (re-aprendizaje) */
void lms_reset(lms_t *f);

/* Cambia mu sin tocar los coeficientes */
void lms_set_mu(lms_t *f, float32_t mu);

/* Procesa una muestra: x = entrada (referencia), d = salida deseada.
 * Devuelve y (salida del filtro) y escribe el error en *e. */
float32_t lms_step(lms_t *f, float32_t x, float32_t d, float32_t *e);

/* Copia los coeficientes en orden natural: w[0] multiplica a x(n) */
void lms_get_w(const lms_t *f, float32_t *w);

const char *lms_alg_name(lms_alg_t alg);

#endif /* LMS_H_ */
```

Código 2: `lms.c`

```
/*
 * TP4 - Procesamiento Digital de Señales
 * lms.c: filtro transversal adaptativo LMS / NLMS (ver lms.h).
 */
#include "lms.h"

#include <string.h>

void lms_init(lms_t *f, lms_alg_t alg, uint16_t L, float32_t mu)
{
    if (L > LMS_MAX_TAPS)
    {
        L = (uint16_t)LMS_MAX_TAPS;
    }
    f->alg = alg;
    f->L   = L;
    f->mu  = mu;
    lms_reset(f);
}

void lms_reset(lms_t *f)
{
    (void)memset(f->x_hist, 0, sizeof(f->x_hist));
    (void)memset(f->w, 0, sizeof(f->w));
    (void)memset(f->cm_coef, 0, sizeof(f->cm_coef));
    (void)memset(f->cm_state, 0, sizeof(f->cm_state));

    /* arm_lms_*_init pone el estado en cero; los coeficientes los pone el usuario */
    arm_lms_init_f32(&f->cm, f->L, f->cm_coef, f->cm_state, f->mu, 1U);
    arm_lms_norm_init_f32(&f->nm, f->L, f->cm_coef, f->cm_state, f->mu, 1U);
}

void lms_set_mu(lms_t *f, float32_t mu)
{
    f->mu    = mu;
    f->cm.mu = mu;
```

```c
    f->nm.mu = mu;
}

/*
 * Rutina de la diapositiva "Implementación en DSP" (segunda versión, la que
 * desplaza la historia, adapta y acumula en un solo lazo). Se mantiene la
 * misma estructura; sólo cambia que el error se calcula ANTES de adaptar,
 * con la salida del filtro de la muestra actual, como en arm_lms_f32. En la
 * versión de la diapositiva el (d - y) del lazo usa la y que se va
 * acumulando, que no es el error de esta muestra.
 */
static float32_t lms_clase_step(lms_t *f, float32_t xin, float32_t d, float32_t *e_out)
{
    const int L     = (int)f->L;
    float32_t *x    = f->x_hist;
    float32_t *w    = f->w;
    float32_t y     = 0.0f;
    int i;

    /* 1) actualizar la historia de la entrada (línea de retardo) */
    for (i = L - 1; i >= 1; i--)
    {
        x[i] = x[i - 1];
    }
    x[0] = xin;

    /* 2) salida del FIR con los coeficientes actuales */
    for (i = L - 1; i >= 0; i--)
    {
        y += w[i] * x[i];
    }

    /* 3) error y adaptación de los coeficientes (LMS) */
    const float32_t e   = d - y;
    const float32_t mue = f->mu * e;
    for (i = L - 1; i >= 0; i--)
    {
        w[i] += mue * x[i];
    }

    *e_out = e;
    return y;
}

float32_t lms_step(lms_t *f, float32_t x, float32_t d, float32_t *e)
{
    float32_t y;

    switch (f->alg)
    {
        case LMS_ALG_CMSIS:
            arm_lms_f32(&f->cm, &x, &d, &y, e, 1U);
            break;
        case LMS_ALG_NLMS:
            arm_lms_norm_f32(&f->nm, &x, &d, &y, e, 1U);
            break;
        case LMS_ALG_CLASE:
        default:
            y = lms_clase_step(f, x, d, e);
            break;
    }
    return y;
}
```

```
100
101 void lms_get_w(const lms_t *f, float32_t *w)
102 {
103     const uint32_t L = f->L;
104
105     if (f->alg == LMS_ALG_CLASE)
106     {
107         (void)memcpy(w, f->w, L * sizeof(float32_t));
108     }
109     else
110     {
111         /* CMSIS guarda los coeficientes en orden invertido: cm_coef[L-1] multiplica a x(n)
    ↪ */
112         for (uint32_t k = 0U; k < L; k++)
113         {
114             w[k] = f->cm_coef[L - 1U - k];
115         }
116     }
117 }
118
119 const char *lms_alg_name(lms_alg_t alg)
120 {
121     switch (alg)
122     {
123         case LMS_ALG_CMSIS:
124             return "CMSIS_LMS";
125         case LMS_ALG_NLMS:
126             return "CMSIS_NLMS";
127         case LMS_ALG_CLASE:
128         default:
129             return "LMS_CLASE";
130     }
131 }
```

Código 3: parte1.c

```
1  /*
2   * TP4 - Procesamiento Digital de Señales
3   * parte1.c: identificación de planta con LMS (ver parte1.h).
4   */
5  #include "parte1.h"
6
7  #include <math.h>
8
9  #include "coefs.h"
10 #include "fir_fijo.h"
11 #include "prng.h"
12
13 static lms_t s_lms;
14 static fir_fijo_t s_planta;
15
16 void p1_default_cfg(p1_cfg_t *cfg)
17 {
18     cfg->alg       = LMS_ALG_CMSIS;
19     cfg->mu        = 0.01f;
20     cfg->input     = P1_IN_RANDOM;
21     cfg->amp       = 1.0f;
22     cfg->sigma_v   = 0.0f;
23     cfg->n_total   = 20000U;
24     cfg->block     = 100U;
25     cfg->seed      = 1U;
26     cfg->send_diff = true;
27 }
```

```c

float p1_input_power(const p1_cfg_t *cfg)
{
    const float a2 = cfg->amp * cfg->amp;
    return (cfg->input == P1_IN_CONST) ? a2 : (a2 / 3.0f);
}

/*
 * Condición de convergencia "segura" de la teoría: 0 < 2mu_teo < 2/tr(R), con
 * tr(R) = L * Px. Con la convención de este código (mu = 2 mu_teo):
 *     LMS : 0 < mu < 2 / (L * Px)
 *     NLMS: 0 < mu < 2 (no depende de la potencia de la entrada)
 * Vale tanto para la entrada aleatoria (R = Px I) como para la constante
 * (R = A^2 * 1 1^T, un solo autovalor no nulo, igual a L A^2 = tr(R)).
 */
float p1_mu_max(const p1_cfg_t *cfg)
{
    if (cfg->alg == LMS_ALG_NLMS)
    {
        return 2.0f;
    }
    return 2.0f / ((float)COEFS_L * p1_input_power(cfg));
}

/*
 * Constante de tiempo de la curva de aprendizaje (MSE), en muestras:
 *   el modo p del error de los pesos decae como (1 - mu lambda_p)^n, y el
 *   MSE como su cuadrado -> tau_mse ~ 1 / (2 mu lambda_p).
 *   Aleatoria blanca: todos los lambda = Px. Constante: lambda = L A^2.
 *   NLMS: el paso efectivo es mu / (L Px).
 */
static float p1_tau(const p1_cfg_t *cfg)
{
    float lambda = (cfg->input == P1_IN_CONST) ? ((float)COEFS_L * p1_input_power(cfg)) :
    ↪ p1_input_power(cfg);
    float mu_ef  = cfg->mu;
    if (cfg->alg == LMS_ALG_NLMS)
    {
        mu_ef = cfg->mu / ((float)COEFS_L * p1_input_power(cfg));
    }
    const float den = 2.0f * mu_ef * lambda;
    return (den > 0.0f) ? (1.0f / den) : 0.0f;
}

static float coef_error(float *diff, const float *w)
{
    float acc = 0.0f;
    for (uint32_t k = 0U; k < COEFS_L; k++)
    {
        diff[k] = g_planta[k] - w[k];
        acc += diff[k] * diff[k];
    }
    return acc;
}

/* 10 log10(x) sin math.h de punto flotante doble (alcanza con ~0,01 dB) */
static float db10(float x)
{
    if (!(x > 0.0f))
    {
        return -999.0f;
    }
    return 10.0f * 0.30102999566f * log2f(x);
```

```
}

p1_status_t p1_run(const p1_cfg_t *cfg, const p1_io_t *io)
{
    txt_line_t ln;
    float w[COEFS_L];
    float diff[COEFS_L];
    prng_t rx;
    prng_t rv;
    p1_status_t st   = P1_OK;
    uint32_t cyc_acc = 0U;
    uint32_t n       = 0U;
    float mse        = 0.0f;
    float werr2      = 0.0f;
    const float sv   = cfg->sigma_v * 1.7320508f; /* uniforme [-a, a] tiene desvío a/sqrt(3) */

    lms_init(&s_lms, cfg->alg, (uint16_t)COEFS_L, cfg->mu);
    fir_fijo_init(&s_planta, g_planta, COEFS_L);
    prng_seed(&rx, cfg->seed);
    prng_seed(&rv, cfg->seed ^ 0xA5A5A5A5U);

    /* Encabezado del experimento */
    txt_begin(&ln, "P1I");
    txt_kv_str(&ln, "alg", lms_alg_name(cfg->alg));
    txt_kv_f32(&ln, "mu", cfg->mu);
    txt_kv_str(&ln, "in", (cfg->input == P1_IN_CONST) ? "CONST" : "RANDOM");
    txt_kv_f32(&ln, "A", cfg->amp);
    txt_kv_f32(&ln, "Px", p1_input_power(cfg));
    txt_kv_u32(&ln, "L", COEFS_L);
    txt_kv_u32(&ln, "N", cfg->n_total);
    txt_kv_u32(&ln, "blk", cfg->block);
    txt_kv_f32(&ln, "sv", cfg->sigma_v);
    txt_kv_u32(&ln, "seed", cfg->seed);
    txt_kv_f32(&ln, "mu_max", p1_mu_max(cfg));
    txt_kv_f32(&ln, "tau", p1_tau(cfg));
    txt_end(&ln, io->write);

    txt_begin(&ln, "P1P");
    for (uint32_t k = 0U; k < COEFS_L; k++)
    {
        txt_f32(&ln, g_planta[k]);
    }
    txt_end(&ln, io->write);

    while (n < cfg->n_total)
    {
        float acc = 0.0f;
        uint32_t b;

        /* --- un bloque de 'block' muestras --- */
        for (b = 0U; (b < cfg->block) && (n < cfg->n_total); b++, n++)
        {
            /* entrada */
            const float x = (cfg->input == P1_IN_CONST) ? cfg->amp : (cfg->amp * prng_uniform(&rx));

            /* planta (+ ruido de medición) -> salida deseada */
            float d = fir_fijo_step(&s_planta, x);
            if (sv > 0.0f)
            {
                d += sv * prng_uniform(&rv);
            }
```

```
151
152             /* procesador adaptativo */
153             float e;
154             const uint32_t c0 = (io->cycles != NULL) ? io->cycles() : 0U;
155             (void)lms_step(&s_lms, x, d, &e);
156             if (io->cycles != NULL)
157             {
158                 cyc_acc += io->cycles() - c0;
159             }
160             acc += e * e;
161         }
162
163         /* --- cada bloque: MSE promedio y error de los coeficientes --- */
164         mse = acc / (float)b;
165         lms_get_w(&s_lms, w);
166         werr2 = coef_error(diff, w);
167
168         txt_begin(&ln, "P1M");
169         txt_u32(&ln, n);
170         txt_f32(&ln, mse);
171         txt_f32(&ln, werr2);
172         txt_f32(&ln, db10(mse));
173         txt_end(&ln, io->write);
174
175         if (cfg->send_diff)
176         {
177             txt_begin(&ln, "P1D");
178             txt_u32(&ln, n);
179             for (uint32_t k = 0U; k < COEFS_L; k++)
180             {
181                 txt_f32(&ln, diff[k]);
182             }
183             txt_end(&ln, io->write);
184         }
185
186         /* Si el MSE explota (mu demasiado grande) no tiene sentido seguir */
187         if (!(mse < 1.0e12f) || !(werr2 < 1.0e12f))
188         {
189             st = P1_DIVERGE;
190             break;
191         }
192         if ((io->abort != NULL) && io->abort())
193         {
194             st = P1_CORTADO;
195             break;
196         }
197     }
198
199     txt_begin(&ln, "P1W");
200     for (uint32_t k = 0U; k < COEFS_L; k++)
201     {
202         txt_f32(&ln, w[k]);
203     }
204     txt_end(&ln, io->write);
205
206     txt_begin(&ln, "P1F");
207     txt_u32(&ln, n);
208     txt_f32(&ln, mse);
209     txt_f32(&ln, werr2);
210     txt_u32(&ln, (n > 0U) ? (cyc_acc / n) : 0U);
211     txt_str(&ln, (st == P1_OK) ? "OK" : ((st == P1_DIVERGE) ? "DIVERGE" : "CORTADO"));
212     txt_end(&ln, io->write);
213
```

```
214     return st;
215 }
```

Código 4: parte2.c

```c
/*
 * TP4 - Procesamiento Digital de Señales
 * parte2.c: cancelador adaptativo de ruido (ver parte2.h).
 */
#include "parte2.h"

#include <math.h>
#include <string.h>

#include "coefs.h"
#include "fir_fijo.h"
#include "prng.h"

static p2_cfg_t s_cfg;
static lms_t s_lms;
static fir_fijo_t s_corr;
static prng_t s_rng;
static float s_w_opt[COEFS_L];

/* Acumuladores del bloque en curso y último bloque terminado */
static uint32_t s_n;
static uint32_t s_b;
static float s_acc_noise;
static float s_acc_res;
static float s_acc_sig;
static p2_stats_t s_last;

void p2_default_cfg(p2_cfg_t *cfg)
{
    cfg->alg       = LMS_ALG_CMSIS;
    cfg->mu        = 0.005f;
    cfg->noise_amp = 0.4f;
    cfg->topo      = P2_CORR_EN_RUIDO;
    cfg->seed      = 1U;
    cfg->block     = 100U;
}

static void compute_w_opt(void)
{
    (void)memset(s_w_opt, 0, sizeof(s_w_opt));
    if (s_cfg.topo == P2_CORR_EN_RUIDO)
    {
        /* Wiener: W = C */
        (void)memcpy(s_w_opt, g_correlador, sizeof(s_w_opt));
    }
    else
    {
        /* W = C^-1 con C(z) ~ c0 / (1 - a z^-1): W(z) ~ (1 - a z^-1) / c0 */
        const float c0 = g_correlador[0];
        const float a  = g_correlador[1] / g_correlador[0];
        s_w_opt[0]     = 1.0f / c0;
        s_w_opt[1]     = -a / c0;
    }
}

void p2_init(const p2_cfg_t *cfg)
{
    s_cfg = *cfg;
```

```
    if (s_cfg.block == 0U)
    {
        s_cfg.block = 100U;
    }
    lms_init(&s_lms, s_cfg.alg, (uint16_t)COEFS_L, s_cfg.mu);
    fir_fijo_init(&s_corr, g_correlador, COEFS_L);
    prng_seed(&s_rng, s_cfg.seed);
    compute_w_opt();

    s_n         = 0U;
    s_b         = 0U;
    s_acc_noise = 0.0f;
    s_acc_res   = 0.0f;
    s_acc_sig   = 0.0f;
    (void)memset(&s_last, 0, sizeof(s_last));
}

const p2_cfg_t *p2_get_cfg(void)
{
    return &s_cfg;
}

void p2_reset_w(void)
{
    lms_reset(&s_lms);
}

void p2_set_mu(float mu)
{
    s_cfg.mu = mu;
    lms_set_mu(&s_lms, mu);
}

bool p2_process(float s, float out[P2_OUT_CNT])
{
    /* 1) ruido aleatorio y su versión correlacionada */
    const float r  = s_cfg.noise_amp * prng_uniform(&s_rng);
    const float rc = fir_fijo_step(&s_corr, r);

    float noise; /* ruido que contamina la señal */
    float ref;   /* referencia del procesador adaptativo */
    if (s_cfg.topo == P2_CORR_EN_RUIDO)
    {
        noise = rc;
        ref   = r;
    }
    else
    {
        noise = r;
        ref   = rc;
    }

    /* 2) señal + ruido y cancelador LMS */
    const float d = s + noise;
    float e;
    const float y = lms_step(&s_lms, ref, d, &e);

    const float res = e - s;
    out[P2_OUT_D]   = d;
    out[P2_OUT_E]   = e;
    out[P2_OUT_Y]   = y;
    out[P2_OUT_S]   = s;
    out[P2_OUT_RES] = 4.0f * res;
```

```
122
123     /* 3) estadísticas por bloque */
124     s_acc_noise += noise * noise;
125     s_acc_res += res * res;
126     s_acc_sig += s * s;
127     s_n++;
128     if (++s_b < s_cfg.block)
129     {
130         return false;
131     }
132
133     const float inv = 1.0f / (float)s_b;
134     s_last.n        = s_n;
135     s_last.p_noise  = s_acc_noise * inv;
136     s_last.p_res    = s_acc_res * inv;
137     s_last.p_sig    = s_acc_sig * inv;
138     s_b             = 0U;
139     s_acc_noise     = 0.0f;
140     s_acc_res       = 0.0f;
141     s_acc_sig       = 0.0f;
142
143     /* Error de los coeficientes respecto del óptimo (sólo cada bloque: es caro) */
144     float w[COEFS_L];
145     float acc = 0.0f;
146     lms_get_w(&s_lms, w);
147     for (uint32_t k = 0U; k < COEFS_L; k++)
148     {
149         const float dk = s_w_opt[k] - w[k];
150         acc += dk * dk;
151     }
152     s_last.werr2 = acc;
153     return true;
154 }
155
156 void p2_get_stats(p2_stats_t *st)
157 {
158     *st = s_last;
159 }
160
161 void p2_get_w(float *w)
162 {
163     lms_get_w(&s_lms, w);
164 }
165
166 void p2_get_w_opt(float *w_opt)
167 {
168     (void)memcpy(w_opt, s_w_opt, sizeof(s_w_opt));
169 }
170
171 const char *p2_out_name(p2_out_t o)
172 {
173     static const char *const names[P2_OUT_CNT] = {"d(senal+ruido)", "e(salida)", "y(
    ↪ ruido_estimado)",
174                                                   "s(senal)", "residuo_x4"};
175     return (o < P2_OUT_CNT) ? names[o] : "?";
176 }
177
178 static float db10(float x)
179 {
180     if (!(x > 0.0f))
181     {
182         return -999.0f;
183     }
```

```
184     return 10.0f * 0.30102999566f * log2f(x);
185 }
186 
187 void p2_print_header(txt_write_fn wr, float fs)
188 {
189     txt_line_t ln;
190     /* Potencia de la referencia: uniforme -> a^2/3; si pasa por C, sum(c^2) = 1 */
191     const float px = s_cfg.noise_amp * s_cfg.noise_amp / 3.0f;
192     float mu_max   = 2.0f / ((float)COEFS_L * px);
193     float tau      = 1.0f / (2.0f * s_cfg.mu * px);
194     if (s_cfg.alg == LMS_ALG_NLMS)
195     {
196         mu_max = 2.0f;
197         tau    = (float)COEFS_L / (2.0f * s_cfg.mu);
198     }
199 
200     txt_begin(&ln, "P2I");
201     txt_kv_str(&ln, "alg", lms_alg_name(s_cfg.alg));
202     txt_kv_f32(&ln, "mu", s_cfg.mu);
203     txt_kv_f32(&ln, "ruido", s_cfg.noise_amp);
204     txt_kv_str(&ln, "topo", (s_cfg.topo == P2_CORR_EN_RUIDO) ? "RUIDO" : "REFERENCIA");
205     txt_kv_f32(&ln, "fs", fs);
206     txt_kv_u32(&ln, "blk", s_cfg.block);
207     txt_kv_f32(&ln, "mu_max", mu_max);
208     txt_kv_f32(&ln, "tau", tau);
209     txt_end(&ln, wr);
210 }
211 
212 void p2_print_stats(txt_write_fn wr, const p2_stats_t *st)
213 {
214     txt_line_t ln;
215     txt_begin(&ln, "P2M");
216     txt_u32(&ln, st->n);
217     txt_f32(&ln, st->p_noise);
218     txt_f32(&ln, st->p_res);
219     txt_f32(&ln, db10(st->p_noise) - db10(st->p_res));
220     txt_f32(&ln, st->werr2);
221     txt_f32(&ln, db10(st->p_sig) - db10(st->p_noise));
222     txt_f32(&ln, db10(st->p_sig) - db10(st->p_res));
223     txt_end(&ln, wr);
224 }
225 
226 void p2_print_w(txt_write_fn wr)
227 {
228     txt_line_t ln;
229     float w[COEFS_L];
230 
231     p2_get_w(w);
232     txt_begin(&ln, "P2W");
233     for (uint32_t k = 0U; k < COEFS_L; k++)
234     {
235         txt_f32(&ln, w[k]);
236     }
237     txt_end(&ln, wr);
238 
239     txt_begin(&ln, "P2O");
240     for (uint32_t k = 0U; k < COEFS_L; k++)
241     {
242         txt_f32(&ln, s_w_opt[k]);
243     }
244     txt_end(&ln, wr);
245 }
```

Código 5: `coefs.c`

```c
/*
 * TP4 - Procesamiento Digital de Señales
 * coefs.c: coeficientes fijos (generados en tools/host_sim/gen_coefs.py).
 */
#include "coefs.h"

/* p[k] = 0,8 e^(-k/8) sin(0,3 pi (k+1)) + 0,05 cos(0,9 k), redondeado a 4 decimales */
const float g_planta[COEFS_L] = {
    0.6972f,  0.7025f,  0.1812f,  -0.3684f, -0.5301f, -0.2622f, 0.1485f,  0.3672f,
    0.2685f,  -0.0122f, -0.2310f, -0.2368f, -0.0649f, 0.1250f,  0.1890f,  0.1019f,
    -0.0464f, -0.1368f, -0.1123f, -0.0089f, 0.0861f,  0.1051f,  0.0449f,  -0.0403f,
    -0.0861f, -0.0643f, 0.0015f,  0.0597f,  0.0694f,  0.0284f,
};

/* c[k] = 0,6 * 0,8^k, redondeado a 4 decimales */
const float g_correlador[COEFS_L] = {
    0.6000f, 0.4800f, 0.3840f, 0.3072f, 0.2458f, 0.1966f, 0.1573f, 0.1258f,
    0.1007f, 0.0805f, 0.0644f, 0.0515f, 0.0412f, 0.0330f, 0.0264f, 0.0211f,
    0.0169f, 0.0135f, 0.0108f, 0.0086f, 0.0069f, 0.0055f, 0.0044f, 0.0035f,
    0.0028f, 0.0023f, 0.0018f, 0.0015f, 0.0012f, 0.0009f,
};
```

Código 6: `fir_fijo.h`

```c
/*
 * TP4 - Procesamiento Digital de Señales
 * fir_fijo.h: FIR de coeficientes fijos, muestra a muestra.
 *
 * Se usa para la PLANTA a identificar (parte 1) y para el CORRELADOR del
 * ruido (parte 2). Los dos son FIR de 30 coeficientes fijos:
 *
 *     y(n) = sum_{k=0}^{L-1} h[k] x(n-k)
 *
 * Los coeficientes están en orden natural (h[0] multiplica a x(n)), igual
 * que los que muestra la placa por el puerto serie y los que usa MATLAB.
 */
#ifndef FIR_FIJO_H_
#define FIR_FIJO_H_

#include <stdint.h>
#include <string.h>

#define FIR_FIJO_MAX 32U

typedef struct
{
    const float *h;
    uint32_t L;
    uint32_t pos;            /* posición de x(n) en el buffer circular */
    float x[FIR_FIJO_MAX];
} fir_fijo_t;

static inline void fir_fijo_init(fir_fijo_t *f, const float *h, uint32_t L)
{
    f->h   = h;
    f->L   = (L > FIR_FIJO_MAX) ? FIR_FIJO_MAX : L;
    f->pos = 0U;
    (void)memset(f->x, 0, sizeof(f->x));
}

static inline float fir_fijo_step(fir_fijo_t *f, float xin)
{
```

```
    uint32_t i = (f->pos == 0U) ? (f->L - 1U) : (f->pos - 1U);
    f->pos     = i;
    f->x[i]    = xin;

    float y = 0.0f;
    for (uint32_t k = 0U; k < f->L; k++)
    {
        y += f->h[k] * f->x[i];          /* h[k] * x(n-k) */
        i = (i + 1U == f->L) ? 0U : (i + 1U);
    }
    return y;
}

#endif /* FIR_FIJO_H_ */
```

Código 7: prng.h

```
/*
 * TP4 - Procesamiento Digital de Señales
 * prng.h: generador pseudoaleatorio xorshift32 (Marsaglia).
 *
 * Se usa uno propio en lugar de rand() para que la secuencia sea idéntica en
 * la placa y en la simulación de la PC, y para poder repetir un experimento
 * con la misma semilla cambiando sólo mu.
 */
#ifndef PRNG_H_
#define PRNG_H_

#include <stdint.h>

typedef struct
{
    uint32_t s;
} prng_t;

static inline void prng_seed(prng_t *p, uint32_t seed)
{
    p->s = (seed != 0U) ? seed : 0x12345678U; /* el estado nunca puede ser 0 */
}

static inline uint32_t prng_u32(prng_t *p)
{
    uint32_t x = p->s;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    p->s = x;
    return x;
}

/* Uniforme en [-1, 1): media 0, varianza 1/3 */
static inline float prng_uniform(prng_t *p)
{
    /* 24 bits de mantisa: (u >> 8) en [0, 2^24) */
    return (float)(int32_t)((prng_u32(p) >> 8) - 0x800000U) * (1.0f / 8388608.0f);
}

#endif /* PRNG_H_ */
```

Código 8: main.c

```
/*
 * TP4 / Laboratorio 4 - Procesamiento Digital de Señales
```

```c
 * Filtros adaptativos (LMS) en la FRDM-MCXN947 (Cortex-M33 con FPU y DSP).
 *
 * PARTE 1 - Identificación de planta (parte1.c). No usa ADC ni DAC: la placa
 *   simula la planta (FIR de 30 coeficientes fijos), la entrada (aleatoria o
 *   constante) y el filtro adaptativo LMS de 30 coeficientes. Cada 100
 *   muestras manda por el puerto serie el MSE promedio y la diferencia entre
 *   los coeficientes de la planta y los del filtro.
 *
 * PARTE 2 - Cancelación de ruido (parte2.c). CTIMER0 marca fs; en cada tick
 *   se dispara el ADC y en la ISR del ADC: s = senoidal del generador,
 *   se le suma ruido pseudoaleatorio pasado por el FIR correlador, el LMS
 *   cancela el ruido y por el DAC sale la señal elegida para el osciloscopio.
 *
 * Teclas:
 *   SW2          Parte 1: arranca un experimento (o lo corta si está corriendo)
 *                Parte 2: pone los coeficientes en cero (re-aprende) + pulso en TRIG
 *   SW3 corto    Parte 1: siguiente mu de la lista
 *                Parte 2: cambia la señal del DAC: d -> e -> y -> s -> residuo
 *   SW3 largo    cambia de Parte 1 a Parte 2 y viceversa
 * LEDs: verde = parte 1, rojo = parte 2, azul = cambia con cada bloque enviado.
 *
 * Comandos por el puerto serie (una letra + valor, terminados en Enter).
 * Escribir ? para ver la lista y la configuración actual.
 */
#include <stdbool.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>

#include "fsl_common.h"
#include "fsl_clock.h"
#include "fsl_ctimer.h"
#include "fsl_dac.h"
#include "fsl_gpio.h"
#include "fsl_lpadc.h"
#include "fsl_lpuart.h"
#include "fsl_spc.h"
#include "fsl_vref.h"

#include "clock_config.h"

#include "arm_math.h"

#include "app_config.h"
#include "app_pins.h"
#include "coefs.h"
#include "lms.h"
#include "parte1.h"
#include "parte2.h"
#include "txt_out.h"

/*******************************************************************************
 * Variables
 ******************************************************************************/
static const float s_p1_mu_list[] = APP_P1_MU_LIST;
#define P1_MU_CNT (sizeof(s_p1_mu_list) / sizeof(s_p1_mu_list[0]))

static volatile uint32_t s_part = 1U;   /* 1 o 2 */

/* Parte 1 */
static p1_cfg_t s_p1;
static uint32_t s_p1_mu_idx = 1U;       /* 0,01 */
static bool s_p1_start;
```

```c
static bool s_p1_sweep;

/* Parte 2 */
static p2_cfg_t s_p2;
static uint32_t s_p2_fs = APP_P2_FS_DEFAULT;
static float s_p2_fs_real;
static volatile p2_out_t s_dac_sel = P2_OUT_E;
static volatile float s_dc        = 1.65f;
static volatile bool s_p2_running;
static volatile bool s_p2_reset_req;
static volatile uint32_t s_trig_cnt;
static bool s_p2_print = true;
static uint32_t s_p2_print_every = 1U;
static uint32_t s_p2_print_cnt;

/* Bloques de estadísticas de la ISR hacia el lazo principal (FIFO) */
#define STATS_FIFO 16U
static p2_stats_t s_stats_fifo[STATS_FIFO];
static volatile uint32_t s_stats_wr;
static volatile uint32_t s_stats_rd;
static volatile uint32_t s_stats_lost;

/* Tiempo de procesamiento por muestra (ISR), en ciclos */
static volatile uint32_t s_cyc_max;
static volatile uint32_t s_cyc_acc;
static volatile uint32_t s_cyc_n;

/* SysTick y teclas */
static volatile uint32_t s_ms;
static volatile bool s_ev_sw2;
static volatile bool s_ev_sw3_short;
static volatile bool s_ev_sw3_long;

/* UART */
static lpuart_handle_t s_uart_handle;
static uint8_t s_uart_ring[64];
static char s_cmd[24];
static uint32_t s_cmd_len;

/*******************************************************************************
 * Utilidades
 ******************************************************************************/
static uint32_t cycles_now(void)
{
    return DWT->CYCCNT;
}

static void cycle_counter_init(void)
{
#if defined(DCB)
    DCB->DEMCR |= DCB_DEMCR_TRCENA_Msk;
#else
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
#endif
    DWT->CYCCNT = 0U;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void uart_write(const char *s, size_t len)
{
    (void)LPUART_WriteBlocking(APP_UART, (const uint8_t *)s, len);
}

```

```
129 static void uart_puts(const char *s)
130 {
131     uart_write(s, strlen(s));
132 }
133
134 static void leds_update(void)
135 {
136     /* activos en bajo */
137     GPIO_PinWrite(APP_LED_GREEN_GPIO, APP_LED_GREEN_PIN, (s_part == 1U) ? 0U : 1U);
138     GPIO_PinWrite(APP_LED_RED_GPIO, APP_LED_RED_PIN, (s_part == 2U) ? 0U : 1U);
139 }
140
141 /*******************************************************************************
142  * Teclas: antirrebote y pulsación larga en el SysTick (1 ms)
143  ******************************************************************************/
144 typedef struct
145 {
146     GPIO_Type *gpio;
147     uint32_t pin;
148     uint8_t stable; /* 1 = suelta (pull-up), 0 = apretada */
149     uint8_t cnt;
150     uint32_t t_press;
151 } button_t;
152
153 static button_t s_sw2 = {APP_SW2_GPIO, APP_SW2_PIN, 1U, 0U, 0U};
154 static button_t s_sw3 = {APP_SW3_GPIO, APP_SW3_PIN, 1U, 0U, 0U};
155
156 /* 0 nada, 1 se soltó tras pulsación corta, 2 tras pulsación larga */
157 static uint8_t button_tick(button_t *b, uint32_t now)
158 {
159     uint8_t raw = (uint8_t)GPIO_PinRead(b->gpio, b->pin);
160     if (raw == b->stable)
161     {
162         b->cnt = 0U;
163         return 0U;
164     }
165     if (++b->cnt < APP_DEBOUNCE_MS)
166     {
167         return 0U;
168     }
169     b->cnt    = 0U;
170     b->stable = raw;
171     if (raw == 0U)
172     {
173         b->t_press = now;
174         return 0U;
175     }
176     return ((now - b->t_press) >= APP_LONG_PRESS_MS) ? 2U : 1U;
177 }
178
179 void SysTick_Handler(void)
180 {
181     uint32_t now = ++s_ms;
182     if (button_tick(&s_sw2, now) != 0U)
183     {
184         s_ev_sw2 = true;
185     }
186     uint8_t r3 = button_tick(&s_sw3, now);
187     if (r3 == 1U)
188     {
189         s_ev_sw3_short = true;
190     }
191     else if (r3 == 2U)
```

```
    {
        s_ev_sw3_long = true;
    }
}

/*******************************************************************************
 * Periféricos (mismos que el TP3)
 ******************************************************************************/
static void uart_cb(LPUART_Type *base, lpuart_handle_t *handle, status_t status, void *
    ↪ userData)
{
    (void)base;
    (void)handle;
    (void)status;
    (void)userData;
}

static void uart_init(void)
{
    lpuart_config_t cfg;

    CLOCK_SetClkDiv(kCLOCK_DivFlexcom4Clk, 1U);
    CLOCK_AttachClk(kFRO12M_to_FLEXCOMM4);

    LPUART_GetDefaultConfig(&cfg);
    cfg.baudRate_Bps = APP_UART_BAUD;
    cfg.enableTx     = true;
    cfg.enableRx     = true;
    (void)LPUART_Init(APP_UART, &cfg, APP_UART_CLK_FREQ);

    LPUART_TransferCreateHandle(APP_UART, &s_uart_handle, uart_cb, NULL);
    LPUART_TransferStartRingBuffer(APP_UART, &s_uart_handle, s_uart_ring, sizeof(s_uart_ring)
    ↪ );
}

static void adc_init(void)
{
    lpadc_config_t adc_cfg;
    lpadc_conv_command_config_t cmd_cfg;
    lpadc_conv_trigger_config_t trg_cfg;
    vref_config_t vref_cfg;

    CLOCK_SetClkDiv(kCLOCK_DivAdc0Clk, 1U);
    CLOCK_AttachClk(kFRO_HF_to_ADC0);

    /* El VREF tiene que estar prendido: polariza al LPADC */
    SPC_EnableActiveModeAnalogModules(SPC0, kSPC_controlVref);
    VREF_GetDefaultConfig(&vref_cfg);
    vref_cfg.bufferMode = kVREF_ModeBandgapOnly;
    VREF_Init(VREF0, &vref_cfg);

    LPADC_GetDefaultConfig(&adc_cfg);
    adc_cfg.enableAnalogPreliminary = true;
    adc_cfg.referenceVoltageSource  = kLPADC_ReferenceVoltageAlt3; /* VDD_ANA = 3,3 V */
#if defined(FSL_FEATURE_LPADC_HAS_CFG_PWRSEL) && (FSL_FEATURE_LPADC_HAS_CFG_PWRSEL == 1U)
    adc_cfg.powerLevelMode = kLPADC_PowerLevelAlt4;
#endif
    LPADC_Init(APP_ADC, &adc_cfg);

#if defined(FSL_FEATURE_LPADC_HAS_CTRL_CALOFSMODE) && FSL_FEATURE_LPADC_HAS_CTRL_CALOFSMODE
    LPADC_SetOffsetCalibrationMode(APP_ADC, kLPADC_OffsetCalibration16bitMode);
#endif
#if defined(FSL_FEATURE_LPADC_HAS_CTRL_CALOFS) && FSL_FEATURE_LPADC_HAS_CTRL_CALOFS
```

```
253     LPADC_DoOffsetCalibration(APP_ADC);
254 #endif
255 #if defined(FSL_FEATURE_LPADC_HAS_CTRL_CAL_REQ) && FSL_FEATURE_LPADC_HAS_CTRL_CAL_REQ
256     SDK_DelayAtLeastUs(1U, SystemCoreClock);
257     LPADC_DoAutoCalibration(APP_ADC);
258 #endif
259 
260     /* Comando 1: canal A2, single-ended, 16 bits */
261     LPADC_GetDefaultConvCommandConfig(&cmd_cfg);
262     cmd_cfg.channelNumber            = APP_ADC_CHANNEL;
263     cmd_cfg.conversionResolutionMode = kLPADC_ConversionResolutionHigh;
264     LPADC_SetConvCommandConfig(APP_ADC, APP_ADC_CMDID, &cmd_cfg);
265 
266     /* Trigger 0 -> comando 1, por software (lo dispara la ISR del CTIMER) */
267     LPADC_GetDefaultConvTriggerConfig(&trg_cfg);
268     trg_cfg.targetCommandId       = APP_ADC_CMDID;
269     trg_cfg.enableHardwareTrigger = false;
270     LPADC_SetConvTriggerConfig(APP_ADC, 0U, &trg_cfg);
271 
272     LPADC_EnableInterrupts(APP_ADC, kLPADC_FIFO0WatermarkInterruptEnable);
273     NVIC_SetPriority(APP_ADC_IRQn, 1U);
274     (void)EnableIRQ(APP_ADC_IRQn);
275 }
276 
277 static void dac_init(void)
278 {
279     dac_config_t cfg;
280 
281     CLOCK_SetClkDiv(kCLOCK_DivDac0Clk, 1U);
282     CLOCK_AttachClk(kFRO_HF_to_DAC0);
283     SPC_EnableActiveModeAnalogModules(SPC0, kSPC_controlDac0);
284 
285     DAC_GetDefaultConfig(&cfg);
286     cfg.referenceVoltageSource = kDAC_ReferenceVoltageSourceAlt1; /* VDD_ANA = 3,3 V */
287     DAC_Init(APP_DAC, &cfg);
288     DAC_Enable(APP_DAC, true);
289     DAC_SetData(APP_DAC, 2048U);
290 }
291 
292 static inline void dac_write_volts(float v)
293 {
294     float c = (v + APP_DAC_MID_V) * (APP_DAC_FULL_SCALE / APP_ADC_VREF_V);
295     if (c < 0.0f)
296     {
297         c = 0.0f;
298     }
299     else if (c > APP_DAC_FULL_SCALE)
300     {
301         c = APP_DAC_FULL_SCALE;
302     }
303     DAC_SetData(APP_DAC, (uint32_t)(c + 0.5f));
304 }
305 
306 static void gpio_init(void)
307 {
308     const gpio_pin_config_t in       = {kGPIO_DigitalInput, 0U};
309     const gpio_pin_config_t led_off  = {kGPIO_DigitalOutput, 1U};
310     const gpio_pin_config_t test_out = {kGPIO_DigitalOutput, 0U};
311 
312     GPIO_PinInit(APP_SW2_GPIO, APP_SW2_PIN, &in);
313     GPIO_PinInit(APP_SW3_GPIO, APP_SW3_PIN, &in);
314     GPIO_PinInit(APP_LED_RED_GPIO, APP_LED_RED_PIN, &led_off);
315     GPIO_PinInit(APP_LED_GREEN_GPIO, APP_LED_GREEN_PIN, &led_off);
```

```
316     GPIO_PinInit(APP_LED_BLUE_GPIO, APP_LED_BLUE_PIN, &led_off);
317     GPIO_PinInit(APP_PIN_TRIG_GPIO, APP_PIN_TRIG_PIN, &test_out);
318     GPIO_PinInit(APP_PIN_PROC_GPIO, APP_PIN_PROC_PIN, &test_out);
319 }
320
321 /*******************************************************************************
322  * Parte 2: muestreo y procesamiento en interrupciones
323  ******************************************************************************/
324
325 /* CTIMER0 a fs: dispara una conversión del ADC */
326 static void ctimer_match_cb(uint32_t flags)
327 {
328     (void)flags;
329     LPADC_DoSoftwareTrigger(APP_ADC, 1U); /* máscara 1 = trigger 0 */
330 }
331
332 static ctimer_callback_t s_ctimer_cb[] = {ctimer_match_cb};
333
334 static void ctimer_init(void)
335 {
336     ctimer_config_t cfg;
337
338     CLOCK_SetClkDiv(kCLOCK_DivCtimer0Clk, 1U);
339     CLOCK_AttachClk(kFRO_HF_to_CTIMER0);
340
341     CTIMER_GetDefaultConfig(&cfg);
342     CTIMER_Init(APP_CTIMER, &cfg);
343     CTIMER_RegisterCallBack(APP_CTIMER, &s_ctimer_cb[0], kCTIMER_SingleCallback);
344     NVIC_SetPriority(APP_CTIMER_IRQn, 0U);
345 }
346
347 /* Arranca el CTIMER a la fs pedida y devuelve la fs real */
348 static float ctimer_start(uint32_t fs)
349 {
350     ctimer_match_config_t m;
351     const uint32_t clk = CLOCK_GetCTimerClkFreq(APP_CTIMER_ID);
352     uint32_t div       = (clk + fs / 2U) / fs;
353
354     if (div < 2U)
355     {
356         div = 2U;
357     }
358     CTIMER_StopTimer(APP_CTIMER);
359     m.matchValue         = div - 1U;
360     m.enableCounterReset = true;
361     m.enableCounterStop  = false;
362     m.outControl         = kCTIMER_Output_NoAction;
363     m.outPinInitState    = false;
364     m.enableInterrupt    = true;
365     CTIMER_SetupMatch(APP_CTIMER, kCTIMER_Match_0, &m);
366     (void)EnableIRQ(APP_CTIMER_IRQn);
367     CTIMER_Reset(APP_CTIMER);
368     CTIMER_StartTimer(APP_CTIMER);
369
370     return (float)clk / (float)div;
371 }
372
373 static void ctimer_stop(void)
374 {
375     CTIMER_StopTimer(APP_CTIMER);
376 }
377
378 /* Una muestra de la parte 2 por cada conversión del ADC */
```

```
379 void APP_ADC_IRQHandler(void)
380 {
381     lpadc_conv_result_t r;
382     while (LPADC_GetConvResult(APP_ADC, &r, 0U))
383     {
384         if (!s_p2_running)
385         {
386             continue;
387         }
388         GPIO_PortSet(APP_PIN_PROC_GPIO, 1UL << APP_PIN_PROC_PIN);
389         const uint32_t c0 = cycles_now();
390 
391         if (s_p2_reset_req)
392         {
393             s_p2_reset_req = false;
394             p2_reset_w();
395             s_trig_cnt = APP_TRIG_WIDTH;
396             GPIO_PortSet(APP_PIN_TRIG_GPIO, 1UL << APP_PIN_TRIG_PIN);
397         }
398 
399         /* cuentas -> volts, y se saca la continua (offset del generador) */
400         const float v = (float)r.convValue * (APP_ADC_VREF_V / APP_ADC_FULL_SCALE);
401         float dc      = s_dc;
402         dc += (v - dc) * APP_DC_ALPHA;
403         s_dc = dc;
404 
405         float out[P2_OUT_CNT];
406         const bool blk = p2_process(v - dc, out);
407         dac_write_volts(out[s_dac_sel]);
408 
409         if (blk)
410         {
411             const uint32_t w = s_stats_wr;
412             if ((w - s_stats_rd) < STATS_FIFO)
413             {
414                 p2_get_stats(&s_stats_fifo[w % STATS_FIFO]);
415                 s_stats_wr = w + 1U;
416             }
417             else
418             {
419                 s_stats_lost++;
420             }
421         }
422 
423         if (s_trig_cnt > 0U)
424         {
425             if (--s_trig_cnt == 0U)
426             {
427                 GPIO_PortClear(APP_PIN_TRIG_GPIO, 1UL << APP_PIN_TRIG_PIN);
428             }
429         }
430 
431         const uint32_t dc_cyc = cycles_now() - c0;
432         s_cyc_acc += dc_cyc;
433         s_cyc_n++;
434         if (dc_cyc > s_cyc_max)
435         {
436             s_cyc_max = dc_cyc;
437         }
438         GPIO_PortClear(APP_PIN_PROC_GPIO, 1UL << APP_PIN_PROC_PIN);
439     }
440     SDK_ISR_EXIT_BARRIER;
441 }
```

```
442
443 static void p2_start(void)
444 {
445     s_p2_running = false;
446     ctimer_stop();
447     p2_init(&s_p2);
448     s_stats_rd   = s_stats_wr;
449     s_cyc_max    = 0U;
450     s_cyc_acc    = 0U;
451     s_cyc_n      = 0U;
452     s_p2_fs_real = ctimer_start(s_p2_fs);
453     p2_print_header(uart_write, s_p2_fs_real);
454     s_p2_running = true;
455 }
456
457 static void p2_stop(void)
458 {
459     s_p2_running = false;
460     ctimer_stop();
461     DAC_SetData(APP_DAC, 2048U);
462     GPIO_PortClear(APP_PIN_TRIG_GPIO, 1UL << APP_PIN_TRIG_PIN);
463 }
464
465 static void p2_print_timing(void)
466 {
467     txt_line_t ln;
468     const uint32_t n   = s_cyc_n;
469     const uint32_t avg = (n > 0U) ? (s_cyc_acc / n) : 0U;
470     txt_begin(&ln, "P2T");
471     txt_kv_u32(&ln, "cyc_prom", avg);
472     txt_kv_u32(&ln, "cyc_max", s_cyc_max);
473     txt_kv_u32(&ln, "f_cpu", SystemCoreClock);
474     txt_kv_f32(&ln, "us_prom", (float)avg * 1.0e6f / (float)SystemCoreClock);
475     txt_kv_f32(&ln, "us_disp", 1.0e6f / s_p2_fs_real);
476     txt_kv_u32(&ln, "bloques_perdidos", s_stats_lost);
477     txt_end(&ln, uart_write);
478 }
479
480 /*******************************************************************************
481  * Comandos y estado
482  ******************************************************************************/
483 static void print_help(void)
484 {
485     txt_line_t ln;
486
487     uart_puts("\r\n# TP4 PDS - Filtros adaptativos LMS - FRDM-MCXN947\r\n"
488               "# Comandos (letra + valor + Enter):\r\n"
489               "#   1 / 2      pasar a la Parte 1 / Parte 2\r\n"
490               "#   U<mu>      paso de adaptacion (ej. U0.01)\r\n"
491               "#   G<0|1|2>   algoritmo: 0 LMS de clase, 1 arm_lms_f32, 2 arm_lms_norm_f32 (
    ↪ NLMS)\r\n"
492               "#   A<amp>     P1: amplitud de la entrada | P2: amplitud del ruido (V)\r\n"
493               "#   P1: E<0|1> entrada aleatoria/constante, V<sigma> ruido de medicion,\r\n"
494               "#       N<muestras>, R<semilla>, D<0|1> mandar diferencias, S arrancar,\r\n"
495               "#       B barrido de mu, X cortar\r\n"
496               "#   P2: Q<0|1> correlador en el ruido/en la referencia, F<hz> fs,\r\n"
497               "#       O<0..4> DAC: d, e, y, s, residuo; Z reiniciar coeficientes,\r\n"
498               "#       C ver coeficientes, T tiempos, M<k> imprimir 1 de cada k bloques (0 =
    ↪ no)\r\n");
499
500     txt_begin(&ln, "#CFG");
501     txt_kv_u32(&ln, "parte", s_part);
502     if (s_part == 1U)
```

```
503     {
504         txt_kv_str(&ln, "alg", lms_alg_name(s_p1.alg));
505         txt_kv_f32(&ln, "mu", s_p1.mu);
506         txt_kv_str(&ln, "in", (s_p1.input == P1_IN_CONST) ? "CONST" : "RANDOM");
507         txt_kv_f32(&ln, "A", s_p1.amp);
508         txt_kv_f32(&ln, "sv", s_p1.sigma_v);
509         txt_kv_u32(&ln, "N", s_p1.n_total);
510         txt_kv_u32(&ln, "seed", s_p1.seed);
511         txt_kv_f32(&ln, "mu_max", p1_mu_max(&s_p1));
512     }
513     else
514     {
515         txt_kv_str(&ln, "alg", lms_alg_name(s_p2.alg));
516         txt_kv_f32(&ln, "mu", s_p2.mu);
517         txt_kv_f32(&ln, "ruido", s_p2.noise_amp);
518         txt_kv_str(&ln, "topo", (s_p2.topo == P2_CORR_EN_RUIDO) ? "RUIDO" : "REFERENCIA");
519         txt_kv_f32(&ln, "fs", s_p2_fs_real);
520         txt_kv_str(&ln, "dac", p2_out_name(s_dac_sel));
521         txt_kv_f32(&ln, "dc", s_dc);
522     }
523     txt_end(&ln, uart_write);
524 }
525
526 static void set_part(uint32_t part)
527 {
528     if (part == s_part)
529     {
530         return;
531     }
532     s_part = part;
533     if (part == 2U)
534     {
535         p2_start();
536     }
537     else
538     {
539         p2_stop();
540     }
541     leds_update();
542     print_help();
543 }
544
545 static void process_command(const char *cmd)
546 {
547     const char c  = (char)((cmd[0] >= 'a' && cmd[0] <= 'z') ? (cmd[0] - 32) : cmd[0]);
548     const float v = (float)strtod(&cmd[1], NULL);
549     const long iv = strtol(&cmd[1], NULL, 10);
550     bool p2_restart = false;
551
552     switch (c)
553     {
554         case '1':
555             set_part(1U);
556             return;
557         case '2':
558             set_part(2U);
559             return;
560         case '?':
561         case 'H':
562             print_help();
563             return;
564         case 'U':
565             if (v > 0.0f)
```

```
            {
                if (s_part == 1U)
                {
                    s_p1.mu = v;
                }
                else
                {
                    s_p2.mu = v;
                    p2_set_mu(v); /* sin borrar lo aprendido */
                    p2_print_header(uart_write, s_p2_fs_real);
                }
            }
            break;
        case 'G':
            if ((iv >= 0) && (iv <= 2))
            {
                if (s_part == 1U)
                {
                    s_p1.alg = (lms_alg_t)iv;
                }
                else
                {
                    s_p2.alg   = (lms_alg_t)iv;
                    p2_restart = true;
                }
            }
            break;
        case 'A':
            if (v >= 0.0f)
            {
                if (s_part == 1U)
                {
                    s_p1.amp = v;
                }
                else
                {
                    s_p2.noise_amp = v;
                    p2_restart     = true;
                }
            }
            break;
        default:
            break;
    }

    if (s_part == 1U)
    {
        switch (c)
        {
            case 'E':
                s_p1.input = (iv != 0) ? P1_IN_CONST : P1_IN_RANDOM;
                break;
            case 'V':
                s_p1.sigma_v = (v > 0.0f) ? v : 0.0f;
                break;
            case 'N':
                if ((iv >= (long)APP_P1_BLOCK) && (iv <= 1000000L))
                {
                    s_p1.n_total = (uint32_t)iv;
                }
                break;
            case 'R':
                s_p1.seed = (uint32_t)iv;
```

```
629                 break;
630             case 'D':
631                 s_p1.send_diff = (iv != 0);
632                 break;
633             case 'S':
634                 s_p1_start = true;
635                 break;
636             case 'B':
637                 s_p1_sweep = true;
638                 break;
639             default:
640                 break;
641         }
642     }
643     else
644     {
645         switch (c)
646         {
647             case 'Q':
648                 s_p2.topo  = (iv != 0) ? P2_CORR_EN_REFERENCIA : P2_CORR_EN_RUIDO;
649                 p2_restart = true;
650                 break;
651             case 'F':
652                 if ((iv >= (long)APP_FS_MIN) && (iv <= (long)APP_FS_MAX))
653                 {
654                     s_p2_fs    = (uint32_t)iv;
655                     p2_restart = true;
656                 }
657                 break;
658             case 'O':
659                 if ((iv >= 0) && (iv < (long)P2_OUT_CNT))
660                 {
661                     s_dac_sel = (p2_out_t)iv;
662                 }
663                 break;
664             case 'Z':
665                 s_p2_reset_req = true;
666                 p2_print_header(uart_write, s_p2_fs_real); /* marca el reinicio en el
    ↪ registro */
667                 break;
668             case 'C':
669                 p2_print_w(uart_write);
670                 break;
671             case 'T':
672                 p2_print_timing();
673                 break;
674             case 'M':
675                 s_p2_print       = (iv > 0);
676                 s_p2_print_every = (iv > 0) ? (uint32_t)iv : 1U;
677                 break;
678             default:
679                 break;
680         }
681         if (p2_restart)
682         {
683             p2_start();
684         }
685     }
686 }
687
688 static void uart_poll_commands(void)
689 {
690     while (LPUART_TransferGetRxRingBufferLength(APP_UART, &s_uart_handle) > 0U)
```

```
691     {
692         uint8_t ch;
693         size_t got             = 0U;
694         lpuart_transfer_t xfer = {.data = &ch, .dataSize = 1U};
695         (void)LPUART_TransferReceiveNonBlocking(APP_UART, &s_uart_handle, &xfer, &got);
696         if (got == 0U)
697         {
698             break;
699         }
700         if ((ch == '\n') || (ch == '\r'))
701         {
702             if (s_cmd_len > 0U)
703             {
704                 s_cmd[s_cmd_len] = '\0';
705                 process_command(s_cmd);
706                 s_cmd_len = 0U;
707             }
708         }
709         else if (s_cmd_len < (sizeof(s_cmd) - 1U))
710         {
711             s_cmd[s_cmd_len++] = (char)ch;
712         }
713         else
714         {
715             s_cmd_len = 0U;
716         }
717     }
718 }
719
720 static void handle_events(void)
721 {
722     if (s_ev_sw3_long)
723     {
724         s_ev_sw3_long = false;
725         set_part((s_part == 1U) ? 2U : 1U);
726     }
727     if (s_ev_sw2)
728     {
729         s_ev_sw2 = false;
730         if (s_part == 1U)
731         {
732             s_p1_start = true;
733         }
734         else
735         {
736             s_p2_reset_req = true;
737             p2_print_header(uart_write, s_p2_fs_real);
738         }
739     }
740     if (s_ev_sw3_short)
741     {
742         s_ev_sw3_short = false;
743         if (s_part == 1U)
744         {
745             s_p1_mu_idx = (s_p1_mu_idx + 1U) % P1_MU_CNT;
746             s_p1.mu     = s_p1_mu_list[s_p1_mu_idx];
747             print_help();
748         }
749         else
750         {
751             s_dac_sel = (p2_out_t)(((uint32_t)s_dac_sel + 1U) % (uint32_t)P2_OUT_CNT);
752             print_help();
753         }
```

```
    }
    uart_poll_commands();
}

/* Durante un experimento de la parte 1: SW2 o el comando X lo cortan */
static bool p1_abort(void)
{
    bool stop = false;
    if (s_ev_sw2)
    {
        s_ev_sw2 = false;
        stop     = true;
    }
    while (LPUART_TransferGetRxRingBufferLength(APP_UART, &s_uart_handle) > 0U)
    {
        uint8_t ch;
        size_t got             = 0U;
        lpuart_transfer_t xfer = {.data = &ch, .dataSize = 1U};
        (void)LPUART_TransferReceiveNonBlocking(APP_UART, &s_uart_handle, &xfer, &got);
        if ((got != 0U) && ((ch == 'X') || (ch == 'x')))
        {
            stop = true;
        }
    }
    GPIO_PortToggle(APP_LED_BLUE_GPIO, 1UL << APP_LED_BLUE_PIN);
    return stop;
}

static void p1_do_runs(void)
{
    const p1_io_t io = {uart_write, cycles_now, p1_abort};

    if (s_p1_sweep)
    {
        /* Barrido: misma entrada (misma semilla), cambia sólo mu */
        p1_cfg_t c = s_p1;
        uart_puts("P1B,INICIO\r\n");
        for (uint32_t i = 0U; i < P1_MU_CNT; i++)
        {
            c.mu = s_p1_mu_list[i];
            if (p1_run(&c, &io) == P1_CORTADO)
            {
                break;
            }
        }
        uart_puts("P1B,FIN\r\n");
    }
    else
    {
        (void)p1_run(&s_p1, &io);
    }
    s_p1_start = false;
    s_p1_sweep = false;
    GPIO_PortSet(APP_LED_BLUE_GPIO, 1UL << APP_LED_BLUE_PIN); /* apagado */
}

/*******************************************************************************
 * main
 ******************************************************************************/
int main(void)
{
    BOARD_InitBootClocks();
    APP_InitPins();
```

```
817
818     gpio_init();
819     uart_init();
820     adc_init();
821     dac_init();
822     ctimer_init();
823     cycle_counter_init();
824     (void)SysTick_Config(SystemCoreClock / 1000U);
825
826     p1_default_cfg(&s_p1);
827     s_p1.n_total = APP_P1_N_DEFAULT;
828     s_p1.block   = APP_P1_BLOCK;
829     s_p1.mu      = s_p1_mu_list[s_p1_mu_idx];
830     p2_default_cfg(&s_p2);
831     s_p2_fs_real = (float)s_p2_fs;
832
833     leds_update();
834     print_help();
835
836     for (;;)
837     {
838         handle_events();
839
840         if (s_part == 1U)
841         {
842             if (s_p1_start || s_p1_sweep)
843             {
844                 p1_do_runs();
845             }
846         }
847         else
848         {
849             /* Bloques de estadísticas que dejó la ISR */
850             while (s_stats_rd != s_stats_wr)
851             {
852                 p2_stats_t st = s_stats_fifo[s_stats_rd % STATS_FIFO];
853                 s_stats_rd++;
854                 if (s_p2_print && (++s_p2_print_cnt >= s_p2_print_every))
855                 {
856                     s_p2_print_cnt = 0U;
857                     p2_print_stats(uart_write, &st);
858                     GPIO_PortToggle(APP_LED_BLUE_GPIO, 1UL << APP_LED_BLUE_PIN);
859                 }
860             }
861         }
862     }
863 }
```

Código 9: app_config.h

```
1  /*
2   * TP4 - Procesamiento Digital de Señales
3   * Filtros adaptativos (LMS) sobre FRDM-MCXN947
4   *
5   * app_config.h: todos los parámetros "tocables" del laboratorio en un solo lugar.
6   */
7  #ifndef APP_CONFIG_H_
8  #define APP_CONFIG_H_
9
10 #include <stdint.h>
11
12 /* ---------------------------------------------------------------------------
13  * Puerto serie: VCOM del MCU-Link (USB de debug, J17) = LP_FLEXCOMM4
```

```
 * (P1_8 = RX, P1_9 = TX). En la PC aparece como "MCU-Link VCom Port (COMx)".
 * Se puede abrir con cualquier terminal (PuTTY, Tera Term, la terminal de
 * MCUXpresso) o con los scripts de MATLAB de TP4/matlab.
 * ------------------------------------------------------------------------- */
#define APP_UART            LPUART4
#define APP_UART_BAUD       115200U
#define APP_UART_CLK_FREQ   12000000U   /* FRO 12 MHz -> FLEXCOMM4 */

/* -------------------------------------------------------------------------
 * Parte 1: identificación de planta (no usa ADC ni DAC)
 * ------------------------------------------------------------------------- */
#define APP_P1_N_DEFAULT    20000U      /* muestras por experimento (200 bloques) */
#define APP_P1_BLOCK        100U        /* MSE promediado cada 100 muestras (consigna) */
/* Valores de mu que se recorren con SW3 y con el comando B (barrido) */
#define APP_P1_MU_LIST      {0.002f, 0.01f, 0.05f, 0.1f, 0.19f, 0.21f}

/* -------------------------------------------------------------------------
 * Parte 2: cancelador de ruido. ADC0 canal A2 = P4_23 (J8 pin 28).
 * Rango de entrada 0 .. 3,3 V -> la senoidal tiene que venir montada sobre
 * ~1,65 V de continua. La continua se estima y se resta en el firmware.
 * ------------------------------------------------------------------------- */
#define APP_ADC             ADC0
#define APP_ADC_IRQn        ADC0_IRQn
#define APP_ADC_IRQHandler  ADC0_IRQHandler
#define APP_ADC_CHANNEL     2U
#define APP_ADC_CMDID       1U
#define APP_ADC_VREF_V      3.3f
#define APP_ADC_FULL_SCALE  65536.0f    /* 16 bits */
#define APP_DC_ALPHA        (1.0f / 4096.0f) /* seguimiento lento de la continua */

#define APP_P2_FS_DEFAULT   8000U       /* frecuencia de muestreo de la parte 2 */
#define APP_FS_MIN          1000U
#define APP_FS_MAX          48000U

/* Base de tiempo de muestreo: CTIMER0, match 0 + reset (igual que en el TP3) */
#define APP_CTIMER          CTIMER0
#define APP_CTIMER_IRQn     CTIMER0_IRQn
#define APP_CTIMER_ID       0U

/* Salida al osciloscopio: DAC0 (12 bits, 0..3,3 V) en P4_2 = J1 pin 4.
 * La señal elegida (d, e, y, s o residuo) sale centrada en APP_DAC_MID_V. */
#define APP_DAC             DAC0
#define APP_DAC_FULL_SCALE  4095.0f
#define APP_DAC_MID_V       1.65f

/* Pines de medición (salidas digitales 0 / 3,3 V) */
#define APP_PIN_TRIG_GPIO   GPIO1       /* P1_23 = J1 pin 8 : pulso al reiniciar los
    ↪ coeficientes */
#define APP_PIN_TRIG_PIN    23U
#define APP_PIN_PROC_GPIO   GPIO4       /* P4_0  = J2 pin 18: en alto mientras se procesa una
    ↪  muestra */
#define APP_PIN_PROC_PIN    0U
#define APP_TRIG_WIDTH      8U          /* ancho del pulso de TRIG, en muestras */

/* Teclas (activas en bajo) */
#define APP_SW2_GPIO        GPIO0       /* SW2 */
#define APP_SW2_PIN         23U
#define APP_SW3_GPIO        GPIO0       /* SW3 */
#define APP_SW3_PIN         6U
#define APP_LONG_PRESS_MS   700U
#define APP_DEBOUNCE_MS     30U

/* LED RGB (activos en bajo) */
```

```
#define APP_LED_RED_GPIO    GPIO0
#define APP_LED_RED_PIN     10U
#define APP_LED_GREEN_GPIO  GPIO0
#define APP_LED_GREEN_PIN   27U
#define APP_LED_BLUE_GPIO   GPIO1
#define APP_LED_BLUE_PIN    2U

#endif /* APP_CONFIG_H_ */
```

Código 10: tp4_captura.m

```
function datos = tp4_captura(puerto, comandos, fin, archivo, opciones)
%TP4_CAPTURA Manda comandos a la placa y guarda lo que contesta.
%
%   datos = tp4_captura(puerto, comandos, fin, archivo)
%
%   puerto    'COM3' (el MCU-Link VCom Port; ver el Administrador de dispositivos)
%   comandos  cell array de comandos, ej. {'1', 'G1', 'E0', 'A1', 'U0.01', 'S'}
%   fin       cuándo dejar de leer:
%               - texto: se corta al recibir una línea que empieza así
%                 (ej. 'P1F' para una corrida, 'P1B,FIN' para un barrido)
%               - número: segundos de captura (para la Parte 2)
%   archivo   .txt donde se guarda todo lo recibido (opcional, '' = no guarda)
%
%   Durante la captura se grafica en vivo la curva de aprendizaje (Parte 1) o
%   la reducción de ruido (Parte 2). Devuelve lo mismo que tp4_parse_log.
%
%   Ejemplos:
%     d = tp4_captura('COM3', {'1','G1','E0','A1','U0.01','S'}, 'P1F', 'p1_mu0.01.txt');
%     d = tp4_captura('COM3', {'1','G1','E0','A1','B'}, 'P1B,FIN', 'p1_barrido.txt');
%     d = tp4_captura('COM3', {'2','U0.005','Z'}, 10, 'p2_mu0.005.txt');
%
%   Requiere MATLAB R2019b o posterior (serialport).

    if nargin < 4
        archivo = '';
    end
    if nargin < 5
        opciones = struct();
    end
    if ~isfield(opciones, 'baud'), opciones.baud = 115200; end
    if ~isfield(opciones, 'timeout_s'), opciones.timeout_s = 120; end
    if ~isfield(opciones, 'vivo'), opciones.vivo = true; end

    sp = serialport(puerto, opciones.baud, 'Timeout', 2);
    limpiar = onCleanup(@() delete(sp));
    configureTerminator(sp, 'CR/LF', 'LF');
    pause(0.2);
    flush(sp);

    % Si se va a cambiar de parte, primero se manda eso y se espera un poco
    for i = 1:numel(comandos)
        writeline(sp, comandos{i});
        pause(0.15);
    end

    lineas = {};
    t0 = tic;
    if opciones.vivo
        [fig, h] = preparar_grafico();
    end
    n_ult = 0;

```

```
    while true
        if isnumeric(fin) && toc(t0) >= fin
            break;
        end
        if toc(t0) > opciones.timeout_s
            warning('tp4_captura: se cumplió el timeout de %g s', opciones.timeout_s);
            break;
        end
        if sp.NumBytesAvailable == 0
            pause(0.01);
            continue;
        end
        ln = readline(sp);
        if isempty(ln)
            continue;
        end
        ln = strtrim(char(ln));
        lineas{end+1, 1} = ln; %#ok<AGROW>

        if opciones.vivo && (numel(lineas) - n_ult >= 10)
            n_ult = numel(lineas);
            actualizar_grafico(fig, h, lineas);
        end
        if ischar(fin) && strncmp(ln, fin, numel(fin))
            break;
        end
    end

    if ~isempty(archivo)
        fid = fopen(archivo, 'w');
        fprintf(fid, '%s\n', lineas{:});
        fclose(fid);
        fprintf('Guardadas %d líneas en %s\n', numel(lineas), archivo);
    end
    datos = tp4_parse_log(lineas);
    if opciones.vivo
        actualizar_grafico(fig, h, lineas);
    end
end

function [fig, h] = preparar_grafico()
    fig = figure('Name', 'TP4 - captura en vivo', 'NumberTitle', 'off');
    h = axes(fig);
    grid(h, 'on');
    xlabel(h, 'muestra n');
    ylabel(h, 'dB');
    title(h, 'Esperando datos...');
end

function actualizar_grafico(fig, h, lineas)
    if ~isvalid(fig)
        return;
    end
    d = tp4_parse_log(lineas);
    cla(h);
    hold(h, 'on');
    if ~isempty(d.p1)
        for k = 1:numel(d.p1)
            r = d.p1(k);
            if ~isempty(r.n)
                plot(h, r.n, r.mse_dB, 'DisplayName', sprintf('\\mu = %g', r.cfg.mu));
            end
        end
```

```
        ylabel(h, 'MSE (dB)');
        title(h, 'Parte 1 - curva de aprendizaje');
        legend(h, 'show', 'Location', 'northeast');
    elseif ~isempty(d.p2)
        r = d.p2(end);
        if ~isempty(r.n)
            plot(h, r.n, r.red_dB, 'DisplayName', 'reducción de ruido');
            plot(h, r.n, r.snr_out_dB, 'DisplayName', 'SNR a la salida');
        end
        ylabel(h, 'dB');
        title(h, sprintf('Parte 2 - \\mu = %g', r.cfg.mu));
        legend(h, 'show', 'Location', 'southeast');
    end
    hold(h, 'off');
    drawnow limitrate;
end
```